REQUIREMENTS SPECIFICATION

Zümm

Beekeeping Management and Monitoring System

Project	Beekeeping management application (Java mini-project)
Type	Functional and technical requirements specification
Version	1.0
Date	July 13, 2026
Method	Manager-GO – Drawing up a requirements specification

Connected hive & performance dashboards

Document following the standard outline: Context / Objectives / Scope / Functional description of needs / Budget envelope / Schedule

Contents

Version history **6**

1 Context and problem definition **7**
- 1.1 General overview . . . 7
- 1.2 Problem statement . . . 7
- 1.3 Purpose . . . 7

2 Project objectives **8**
- 2.1 Main objective . . . 8
- 2.2 Specific objectives . . . 8
- 2.3 Expected results (success indicators) . . . 8

3 Project scope **9**
- 3.1 Covered domain . . . 9
- 3.2 Business model and management rules . . . 9
- 3.3 Users and access rights . . . 9
- 3.4 Out of scope . . . 10
- 3.5 Future perspectives . . . 10

4 Functional description of needs **11**
- 4.1 Entity management (CRUD) . . . 11
- 4.2 Planning and monitoring of visits . . . 11
  - 4.2.1 Visit report . . . 11
- 4.3 Dashboards . . . 11
  - 4.3.1 Calendar / agents × hives matrix . . . 11
  - 4.3.2 Production dashboard (farmer / owner) . . . 11
  - 4.3.3 Alerts dashboard (manager) . . . 12
  - 4.3.4 Summary dashboard (owner) . . . 12
- 4.4 Performance indicators (automated part: preparation) . . . 12
  - 4.4.1 Predictive analysis and connected monitoring . . . 12
  - 4.4.2 Measurement ingestion protocol . . . 12
  - 4.4.3 Default thresholds and alert logic . . . 13
- 4.5 Summary grid of functions . . . 14
- 4.6 Additional features inspired by market solutions . . . 14
- 4.7 Advanced management functions (inspiration from Apiary Book and BeePlus) . . . 15

4.8 Known limits of existing solutions and requirements to avoid them . . . 16

5 Data dictionary and calculation rules 17
5.1 Type conventions . . . 17
5.2 Data dictionary (inputs) . . . 17
5.3 Results and calculation rules (outputs) . . . 19
5.4 Typical queries . . . 19

6 Detailed use cases 20
6.1 UC1: Plan and approve a visit . . . 20
6.2 UC2: Carry out a visit and fill in the report . . . 20
6.3 UC3: View the production dashboard . . . 20
6.4 UC4: Handle a health alert . . . 22
6.5 UC5: Query the honey quantity via the API . . . 22

7 Design (UML models) 23
7.1 Class diagram . . . 23
7.2 Layered view (package) . . . 23
7.3 Deployment view . . . 23
7.4 Sequence diagram (UC2) . . . 25
7.5 Authentication sequence (Google OAuth) . . . 25
7.6 Object diagram . . . 25
7.7 Activity diagram . . . 25
7.8 State-machine diagram . . . 25
7.9 Collaboration diagram . . . 25
7.10 Component diagram . . . 30

8 Constraints and technical requirements 31
8.1 Architecture . . . 31
8.2 Technical requirements adopted . . . 31
8.3 Non-functional requirements . . . 32
8.4 Design questions to cover . . . 32
8.5 Justification of the technology choices . . . 32
8.6 Packaging and deployment . . . 33

9 Budget envelope and resources 34
9.1 Framework . . . 34
9.2 Mobilised resources . . . 34
9.3 Economic valuation: industrialisation hypothesis . . . 34
9.3.1 Reference workload . . . 34
9.3.2 Breakdown of a reference envelope . . . 35
9.3.3 Sensitivity to the daily rate . . . 35
9.3.4 Adjustable scope . . . 35
9.3.5 Total cost of ownership . . . 35

10 Schedule and deliverables 36

10.1 Expected deliverables . . . 36
10.2 Acceptance criteria . . . 36
10.3 UML diagrams to produce . . . 37
10.4 Provisional schedule . . . 37

11 Test plan and validation **38**
11.1 Test cases . . . 38
11.2 Traceability matrix . . . 39

12 Beekeeping domain reference and good practices **40**
12.1 Honey production factors . . . 40
12.2 Location and configuration of a site . . . 40
12.3 Hive types and reference yield . . . 41
12.4 Classification of hive models . . . 41
12.4.1 Common composition of a frame hive . . . 42
12.5 Colony inspection protocol . . . 42
12.6 Swarming and colony splitting . . . 42
12.7 Well-being indicators and causes of decline . . . 43
12.8 Impact on the configurable thresholds . . . 43

A Appendix: Physical structure of a hive **45**

B Glossary **46**

C References and sources **48**

D Appendix: Technology stack and comparison of alternatives **50**
D.1 Adopted technology stack . . . 50
D.2 Justified comparison of the alternatives . . . 51
D.2.1 Application foundation . . . 51
D.2.2 Data access . . . 51
D.2.3 Database management system . . . 52
D.2.4 Third-party API style . . . 52
D.2.5 User interface . . . 52
D.2.6 Authentication and access control . . . 53
D.2.7 Configuration externalisation . . . 53
D.2.8 Mapping and foraging radii . . . 53
D.2.9 Charting library . . . 53
D.2.10 Offline and mobile access . . . 54
D.2.11 Integration testing . . . 54

E Appendix – SOLID principles and design patterns **55**
E.1 The five SOLID principles . . . 55
E.2 Chosen design patterns . . . 56
E.3 Refactored design view . . . 57
E.4 Dynamic illustration: Observer and Command patterns . . . 57
E.4.1 Observer (+ Strategy): triggering an alert . . . 58

E.4.2 Command: offline mode and deferred synchronisation . . . 58
E.4.3 State: hive life cycle . . . 58
E.4.4 Composite: honey-quantity calculation . . . 61
E.4.5 Strategy: conversion of heterogeneous units . . . 61
E.4.6 Adapter: ingestion of heterogeneous sensors . . . 62
E.5 Distribution of the patterns per layer . . . 62

F Appendix – Complements: data, UI, security and tests 64
F.1 Logical data model (LDM) . . . 64
F.1.1 Vocabulary correspondence table . . . 64
F.2 Interface mockups (wireframes) . . . 65
F.3 Access-rights matrix (RBAC) . . . 65
F.4 Risk register . . . 65
F.5 Protection of personal data . . . 66
F.6 Test strategy . . . 67

G Appendix – Artificial intelligence and predictive analysis 71
G.1 Guiding principles . . . 71
G.2 Mapping of AI uses . . . 71
G.3 Selected case: adaptive anomaly detection . . . 72
G.3.1 Formalisation (without code) . . . 72
G.3.2 Parameters (all configurable via ConfigZumm.ini) . . . 73
G.4 Integration architecture . . . 74
G.4.1 The detector as an interchangeable Strategy . . . 74
G.4.2 The heavy processing as a decoupled microservice . . . 74
G.5 Anticipated uses (specified interfaces, out of development scope) . . . 74
G.6 Ethics, limits and compliance . . . 75

H Appendix – Security and robustness 76
H.1 Synthetic threat model . . . 76
H.2 Defence in depth . . . 77
H.2.1 Perimeter and transport . . . 77
H.2.2 Application . . . 77
H.2.3 Data and operations . . . 77
H.3 Hardening grid . . . 77
H.4 Robustness: load and reliability testing (k6) . . . 78
H.4.1 Types of tests . . . 78
H.4.2 Zümm-specific scenarios . . . 78
H.4.3 Thresholds and verdict . . . 79
H.4.4 Complement to the test plan . . . 79
H.5 Pre-deployment checklist (go-live) . . . 80
H.6 Compliance and consistency with the requirements . . . 80

I Appendix: Operations and service commitments 81
I.1 Service level objectives (SLO) . . . 81
I.2 Backup and restoration . . . 82

I.2.1 Policy . . . 82
I.2.2 Restoration test . . . 82
I.3 Monitoring and alerting . . . 82
I.4 Reversibility . . . 82
I.5 Knowledge transfer . . . 83

Version history

Version	Date	Author	Changes
0.1	July 9, 2026	Project team	Initial structure and transcription of the brief.
0.5	July 11, 2026	Project team	Beekeeping domain reference and illustrations.
1.0	July 13, 2026	Project team	Market features, defect-prevention requirements, data dictionary, use cases, UML design and test plan.

CHAPTER 1

Context and problem definition

1.1 General overview

The **Zümm** project aims to design an information system dedicated to the management and monitoring of beekeeping activities. Modern beekeeping relies on the regular monitoring of numerous hives, often spread across several geographic sites and operated by different beekeepers. Tracking colony health, productivity and maintenance operations represents a significant organisational burden, today handled in a largely manual and fragmented way.

1.2 Problem statement

Without a centralised tool, beekeeping operators face several difficulties:

- no structured traceability of hive visits and inspections
- difficulty in planning and coordinating the work of beekeeping agents
- lack of visibility over production (honey, weight) and over the profitability of each site or hive
- late detection of anomalies (population decline, production drop, unplanned opening of a hive)
- inability to automatically process measurements from heterogeneous sensors.

1.3 Purpose

The system must provide a structured response in two complementary parts:

1. a **manual management** of handling operations, their planning and monitoring (first part, the subject of this project)
2. a **remote, automated supervision** of performance indicators, fed by sensors, intended to be developed as part of a later project but whose integration the system must anticipate.

CHAPTER 2

Project objectives

2.1 Main objective

Provide a **multi-tier, scalable and loosely coupled** application that manages the entire life cycle of hives, from their physical composition to production monitoring, and offers decision-support dashboards to the various user profiles.

2.2 Specific objectives

- Ensure **CRUD** operations (create, read, update, delete) on all business entities.
- Manage the *farmer / farm / site / hive / body or super / frame* hierarchy.
- Plan, approve and track the inspection visits of beekeeping agents.
- Produce structured and actionable **visit reports**.
- Offer filterable **dashboards** (agents × hives calendar, production alerts, health alerts, financial summary).
- Anticipate the integration of **time-stamped and geolocated** measurements from sensors with heterogeneous units.
- Guarantee **internationalisation**, **security** and **interoperability** (web services).

2.3 Expected results (success indicators)

- Support for at least **three languages** (FR, EN, AR) in the demonstration version.
- Automatic detection of **configurable** threshold breaches (production, population).
- Presentation of production data by farm, site and hive as charts.
- Exposure of an API that can be queried by third-party applications (Excel, etc.).

CHAPTER 3

Project scope

3.1 Covered domain

The scope of this requirements specification covers **the first part** of the system: the manual management of operations, their planning and monitoring, as well as the preparation of the architecture for the future integration of automated indicators.

3.2 Business model and management rules

Entity	Management rules
Farmer	May own several farms.
Farm	May include several apiary sites.
Site	Contains one or more hives, has a geographic location (latitude, longitude, altitude, commissioning date, possible relocation / closure dates). A site may host hives belonging to different farmers and different farms.
Hive	Made up of a base compartment (base / body) and at most **5** extension levels (supers).
Body / Super	May hold wax frames installed on identified *slots*. The base/body must contain **at least 1** frame; the maximum capacity of a body, as of a super, is **10 frames**.
Frame	Occupies a single slot in the body or the super.
Beekeeping agent	Carries out the visits and inspections. A hive may be monitored successively by several agents, but it is under the responsibility of **a single agent** at any given time.
Supervising agent	Approves the visit schedule of the hives under their charge.

Composition constraint: 1 hive = 1 mandatory base/body + 0 to 5 supers, each body/super = 1 to 10 frames, one frame per slot.

3.3 Users and access rights

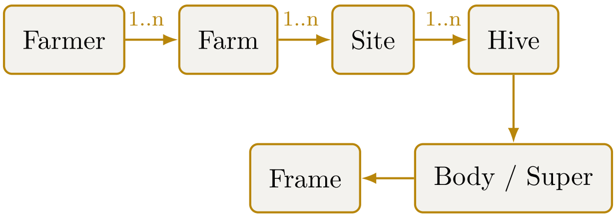

Figure 3.1. Hierarchy of the Zümm system's business entities.

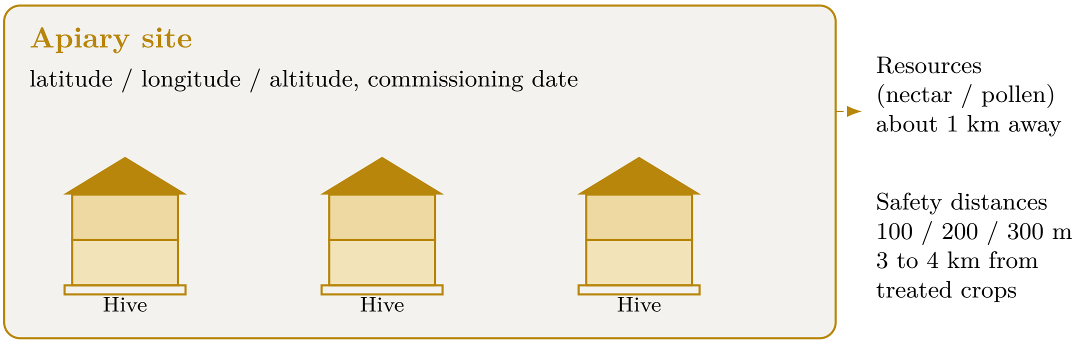

Figure 3.2. Diagram of an apiary site: several geolocated hives and placement constraints.

Profile	Role and permissions
Farmer / Owner	Views the production, profitability and summary dashboards; decides on closing a site or relocating hives.
Beekeeping agent	Carries out the visits, fills in the reports, performs operations on the hives under their responsibility.
Supervising agent	Approves the visit schedules of the hives they are responsible for.
Manager	Monitors the alerts (production or population drop) and triggers imminent inspections.
Administrator	Manages the system parameters, units of measurement, thresholds and reference data.

3.4 Out of scope

- The physical acquisition and hardware deployment of sensors (later project).
- Real-time automation of indicator collection (interface planned but not implemented in this phase).

3.5 Future perspectives

Beyond the scope of this phase, the following evolution may be considered:

- **A separate native mobile application (React Native), distinct from the PWA.** The chosen solution is a PWA, which already provides web/mobile parity and offline operation (see the technologies appendix). A separate native app would only be undertaken if a genuinely native need required it: reliable push notifications on iOS, background geolocation, or distribution through app stores. It would reuse the TypeScript business core and the REST API, but would constitute a second interface codebase to maintain — to be undertaken only on the client's explicit request.

CHAPTER 4

Functional description of needs

4.1 Entity management (CRUD)

The system must ensure the creation, viewing, modification and deletion of all business entities: farmers, farms, sites, hives, bodies/supers, frames, agents, visits and reports, in compliance with the management rules stated in chapter 3.

4.2 Planning and monitoring of visits

- Visits are **regular or irregular**, for various reasons: health inspection, populating with a new swarm, requeening, food replenishment, medication prescription, swarm splitting, adding a frame and/or an extension level, collecting honey frames, production assessment, assessment of the swarm's population and volume.
- Each agent follows a **visit schedule approved** by the supervising agent of the hive concerned.

4.2.1 Visit report

Each visit gives rise to a report containing: **date, time, duration, reason, findings, planned actions, actions carried out, recommendations**, a qualitative assessment of the swarm's population / volume, its health status and its productivity level on a **scale of 1 to 3**.

4.3 Dashboards

4.3.1 Calendar / agents × hives matrix

A matrix cross-referencing **agents and hives**, broken down by month, week, season and year, with **filters** and groupings by site, by agent and by responsible agent. Clicking a cell representing a planned visit displays a **pop-up** summarising what is planned (date, time, reason, actions) and, if the visit has already been carried out, what was done together with the actual date and time.

4.3.2 Production dashboard (farmer / owner)

Highlights the **site × hive** pairs whose production (weight) has exceeded a **configurable threshold**, indicating either an imminent honey harvest or an extension and/or swarm-splitting operation to encourage colony expansion.

4.3.3 Alerts dashboard (manager)

Flags the **site × hive** pairs whose production or population is dropping unusually beyond a configurable threshold, triggering an **alert** and an imminent inspection.

4.3.4 Summary dashboard (owner)

Charts, curves and histograms representing:

- the quantity produced per farm and per site
- the number of operations per farm, per site and per hive
- the **return on investment** (cost / production ratio) to decide the fate of each site or hive (closure, relocation).

4.4 Performance indicators (automated part: preparation)

The measurements are **time-stamped and geolocated**, expressed in **configurable** units (internationalisation of unit systems). The measurements of a single indicator do not necessarily share the same unit, as the sensors are heterogeneous. Indicators concerned:

- weight of the frame, the base and the extension level
- temperature and humidity, inside and outside the hive
- count of bee entry and exit movements
- noise / buzzing level, inside and outside
- light level, inside and outside
- wind speed and direction outside
- hive opening status (operation, storm, theft, animal attack, etc.).

4.4.1 Predictive analysis and connected monitoring

Connected monitoring solutions (precision beekeeping) such as Arnia, BroodMinder or BeeHero show that advanced exploitation of the same indicators makes it possible to anticipate colony events. The data model must remain open to these processes, planned for the automated part.

- acoustic detection of swarming 1 to 5 days in advance through analysis of the colony's sound signature
- confirmation of the queen's presence from the buzzing
- detection of opening, shock or tipping of the hive by accelerometer (theft, fall, animal attack), in addition to the opening indicator
- hive orientation by electronic compass
- continuous weighing by scale to track the nectar flow and production dynamics
- cloud-based analysis by artificial intelligence of acoustic signatures to correlate sound and hive activity.

4.4.2 Measurement ingestion protocol

The automated part remains out of the development scope, but the ingestion interface is specified now so that the data model and the architecture anticipate it.

- **Transport**: a field gateway transmits the measurements to the server, chosen according to connectivity, either by **REST over HTTPS** (`POST /api/mesures`) or by **MQTT** (publishing on the topic `zumm/{rucheId}/{indicateur}`).
- **Format**: **JSON**, a measurement carrying the hive identifier, the indicator type, the value, the unit, the timestamp and the geolocation. A batch upload is an array of measurements.
- **Frequency**: configurable per indicator (for example weight every 15 min, temperature every 5 min, acoustics sampled continuously), in order to control the data volume.
- **Robustness**: **asynchronous** ingestion via a queue, with **idempotency** on the key (hive, indicator, timestamp) to replay without duplicates. In a remote area, local buffering on the gateway then deferred synchronisation.
- **Security**: authenticated gateway (key or certificate), TLS channel. Rejection of values outside the physically plausible range (safeguard).

```
POST /api/mesures        Content-Type: application/json
{
  "rucheId": 42, "typeIndicateur": "TEMPERATURE_INTERNE",
  "valeur": 34.8, "unite": "degC",
  "horodatage": "2026-07-09T14:05:00Z",
  "latitude": 36.806, "longitude": 10.181
}
```

4.4.3 Default thresholds and alert logic

The thresholds are stored in the `SEUIL` table (configurable, chapter 5). The values below are **reference default values**, anchored in the beekeeping protocol (chapter 12).

Indicator	Default threshold	Alert triggered
Internal temperature (brood)	$< 32\,°\mathrm{C}$ or $> 36\,°\mathrm{C}$	Thermal risk to the brood
Internal humidity	$> 80\,\%$	Excess humidity, health risk
Weight: super gain	> production threshold	Imminent honey harvest
Weight: sudden drop	$> 1.5\,\mathrm{kg}$ in less than $1\,\mathrm{h}$	Swarming, theft, fall or attack
Production or population	Relative drop $> 20\,\%$ vs previous period	Imminent health inspection
Entry/exit activity	≈ 0 on a favourable day	Weakened colony or missing queen
Opening status	Open outside a planned visit	Intrusion or disturbance
Acoustic signature	Swarming pattern detected	Swarming pre-alert (1 to 5 days)

Evaluation rule: at each measurement, the value is converted to the reference unit (conversion formula, §5.3), then compared with the configurable threshold. To limit *false alerts*, an alert is only raised if the breach **persists over N consecutive measurements** (debounce / hysteresis).

> Any alert remains a **decision-support aid** confirmed by the visit report: the agent keeps the final validation.

4.5 Summary grid of functions

Function	Description	Priority
Entity CRUD	Complete management of the business model	High
Visit scheduling	Planning + approval by the supervisor	High
Visit reports	Structured recording of findings and actions	High
Agents×hives calendar	Filterable matrix with summary pop-up	High
Production alerts	Detection of threshold breaches	High
Health alerts	Detection of abnormal population/production drops	High
Summary & ROI	Cost/production charts, decision support	Medium
Sensor indicators	Data model ready for automation	Medium
Defect-prevention requirements (drawn from the limits of existing solutions)		
Autonomous deployment	Containerised installation with no mandatory subscription or third-party service	High
Offline mode	Data entry without connection and deferred synchronisation	High
Simple authentication	Google OAuth and fast field-entry flow	High
Configurable modularity	Enabling modules per profile via ConfigZumm.ini	Medium
Data portability	CSV and TXT export, web-service API and backup, with no lock-in	High
Contextual help	Simple, consistent interface, user guidance	Medium
Web and mobile parity	Same functions on all devices	Medium
Internationalisation	FR, EN and AR XML file, extensible to other languages	High
Hardware openness	Heterogeneous sensors and configurable units	Medium
Asynchronous ingestion	Deferred time-stamped measurements, autonomous manual part	Medium
Human validation	Configurable thresholds and confirmation by the visit report	High
Exchange security	SSL and X.509 certificates, authenticated access	High
Decision support	Indicative alerts, the agent keeps the final validation	Medium

4.6 Additional features inspired by market solutions

In order to strengthen usability and value in use, the system draws on the best practices of reference beekeeping applications such as HiveTracks. These features extend the needs already described without replacing them.

Function	Description	Priority
Inspection photos	Attach one or more photos to each visit report for a visual follow-up of the colony and the frames.	High
Assisted voice entry	Dictate the visit report, the transcription automatically feeding the report fields.	Low
Weather context	Display the local weekly weather from the site location to inform operation decisions.	Medium
Map and foraging radii	Position the hives on a map and draw foraging radii (for example 1, 2 and 3 km, configurable units) to visualise the environment accessible to the bees.	Medium
Task list and reminders	Manage a task list and a reminder calendar (feeding, inspection, queen status) so as not to miss any deadline.	High
Queen tracking	Record the history of the queen's status (presence, age, replacement, marking) across visits.	High
Harvest and QR code	Record honey harvests and generate a QR code per batch presenting the tasting notes, the provenance and the photos, for traceability and valorisation.	Medium
Multi-device access	Offer web access and an experience suited to mobile terminals for field entry.	Medium

Consistency with the requirements: the foraging radii and the weather reuse the geolocation of the sites and the configurable-units mechanism. The reminders rely on the existing visit schedule, and the queen tracking enriches the visit report already defined.

4.7 Advanced management functions (inspiration from Apiary Book and BeePlus)

The beekeeping management applications Apiary Book and BeePlus emphasise organisation and traceability functions that complement the system's scope.

Function	Description	Priority
Offline mode and synchronisation	Enter field data without a connection and synchronise it automatically when the network returns.	High
Treatment tracking	Keep the history of the medications and health treatments applied to each hive.	High
Equipment and inventory management	Track the equipment (bodies, supers, frames, hives) and its allocation to sites.	Medium
Financial tracking	Record costs and revenues per farm, site and hive, in support of the return-on-investment calculation.	Medium
Data export	Export the records in CSV and TXT formats for external analysis.	Medium
Disease detection	Help identify diseases from the visit findings.	Low

Function	Description	Priority
Data backup	Back up and restore the data in the cloud.	Medium

Consistency with the requirements: the financial tracking feeds the return-on-investment dashboard, the CSV export extends the web-service API, and the treatment tracking enriches the medication-prescription visit reason.

4.8 Known limits of existing solutions and requirements to avoid them

The analysis of user feedback on market solutions (HiveTracks, Apiary Book, BeePlus) and on connected monitoring systems (Arnia, BroodMinder, BeeHero) brings out recurring limits. The following table sets each limit against the requirement adopted to avoid it in Zümm.

Observed limit	Requirement adopted to avoid it
Paid subscription and paywall	System self-deployable on the operator's own server, with no mandatory subscription; all management functions remain accessible.
Limited offline operation	Robust offline mode with deferred synchronisation; field entry does not require a permanent connection.
Forced account creation and friction	Simple authentication via Google OAuth and a fast entry flow; usability comes first.
Overload of useless features	Modules enabled per profile and via the ConfigZumm.ini configuration file, thanks to the loosely coupled architecture.
Dependency and data loss	Data controlled by the user, CSV and TXT export, web-service API and backup, with no proprietary lock-in.
Dense interface and learning curve	Simple, consistent and user-friendly interface with contextual help, in line with the usability requirements.
Uneven functions across platforms	Functional parity between web and mobile access thanks to the multi-tier architecture.
English-only solutions	Internationalisation through an XML file, at least FR, EN and AR, open to other languages.
High cost of sensor hardware	Openness to heterogeneous and low-cost sensors with configurable units, with no hardware lock-in.
Autonomy and connectivity in a remote area	Asynchronous ingestion of time-stamped measurements; the manual part works without sensors.
False alerts and sensor reliability	Configurable thresholds and human validation through the visit report; alerts remain a decision-support aid.
Data confidentiality and security	SSL encryption and X.509 certificates, authenticated access.
Over-reliance on automation	Positioned as decision support; the beekeeping agent keeps the final validation.

CHAPTER 5

Data dictionary and calculation rules

This chapter answers the question of the system's inputs and outputs. It defines the data handled, their types and the formulas used to compute the results.

5.1 Type conventions

The types used are: integer, decimal, text, date, time, datetime, boolean, enumeration and reference (foreign key to another entity). Physical quantities are associated with a configurable unit to allow the internationalisation of measurement systems.

5.2 Data dictionary (inputs)

Entity	Attribute	Type	Description
Farmer	id, nom, contact	integer, text	Operating owner.
Farm	id, nom, fermier	integer, text, reference	Operation attached to a farmer.
Site	id, nom	integer, text	Beekeeping location.
Site	latitude, longitude	decimal (degrees)	Geolocation.
Site	altitude	decimal (configurable unit)	Site altitude.
Site	dateMiseEnOeuvre, dateDemenagement, dateCloture	date	Site life cycle, the last two optional.
Hive	id, modele	integer, text	Hive hosted by a site.
Hive	nbHausses	integer (0 to 5)	Number of extension levels.
Hive	ferme	reference	Owning farm (distinct from the location site).
Hive	etat	enumeration	Life-cycle state (created, populated, active, splitting, harvesting, closed), see figure 7.8.
Hive	agentResponsable	reference	Agent responsible at the given time.

Entity	Attribute	Type	Description
Compartment	id, type	integer, enumeration	Hive compartment: body or super.
Compartment	capaciteCadres	integer (1 to 10)	Number of frame slots.
Frame	id, slot, type	integer, enumeration	Frame installed on an identified slot.
Frame	poids	decimal (configurable unit)	Frame weight, basis of the honey-quantity calculation.
Agent	id, nom, role	integer, text, enumeration	Beekeeper, supervisor, manager or administrator.
Schedule	id, ruche, agent	integer, reference	Visit schedule.
Schedule	datePrevue, statutApprobation	date, enumeration	Planned date and supervisor approval.
Schedule	superviseur	reference	Supervising agent who approves the schedule.
Visit	id, ruche, agent	integer, reference	Visit carried out.
Visit	planning	reference	Approved schedule behind the visit (optional).
Visit	date, heure, duree	date, time, integer (min)	Timestamp and duration.
Visit	raison	enumeration	Reason for the visit.
Visit	constatations, actionsPrevues, actionsEffectuees, recommandations	text	Report content.
Visit	effectifQualitatif, etatSante	enumeration	Swarm assessment.
Visit	productivite	integer (1 to 3)	Productivity level.
Photo	id, url, visite	integer, text, reference	Photo attached to a visit report.
Measurement	id, ruche, typeIndicateur	integer, reference, enumeration	Measurement of an indicator.
Measurement	valeur, unite	decimal, text	Value and configurable unit.
Measurement	horodatage, latitude, longitude	datetime, decimal	Time-stamped and geolocated measurement.
Harvest	id, ruche, date	integer, reference, date	Honey harvest.
Harvest	quantiteMiel, lotQR	decimal (configurable unit), text	Quantity and batch identifier.

Entity	Attribute	Type	Description
Threshold	id, type, valeur, unite	integer, enumeration, decimal, text	Configurable system threshold.

Reference schema: the dictionary above is the business view of the data. Its normalised relational translation (keys, SQL types and `CHECK` constraints) constitutes the **logical data model** (LDM), which is authoritative for the implementation and is presented in appendix F (figure F.1). The name correspondence between the dictionary, the class diagram and the LDM is given by table F.1.

5.3 Results and calculation rules (outputs)

- **Honey quantity of a hive**: sum of the weights of the honey frames of the supers, minus the tare of the elements.

$$\text{HoneyQuantity}(h) = \sum_{f \in \text{honeyFrames}(h)} \text{weight}(f) - \text{tare}(h)$$

This value is exposed by the web service via `getZummHoneyActualQuantity(zummID)`.

- **Return on investment** of a site or a hive:

$$\text{ROI} = \frac{\text{Valued production}}{\text{Operation costs}}$$

- **Harvest alert**: triggered if production exceeds the configurable threshold.

$$\text{Production}(h) > \text{ProductionThreshold} \Rightarrow \text{harvest or extension alert}$$

- **Drop alert**: triggered if the relative drop exceeds the threshold.

$$\frac{V_{\text{prev}} - V_{\text{current}}}{V_{\text{prev}}} > \text{DropThreshold} \Rightarrow \text{inspection alert}$$

- **Unit conversion** to the reference unit, to compare heterogeneous measurements:

$$V_{\text{reference}} = V_{\text{measurement}} \times \text{conversionFactor}(\text{unit})$$

5.4 Typical queries

The results rely on queries executed via jOOQ, for example for the honey quantity of a hive: `SELECT SUM(quantiteMiel) FROM recolte WHERE ruche = zummID`.

CHAPTER 6

Detailed use cases

This part describes how the system carries out the users' operations. The actors are the farmer, the beekeeping agent, the supervising agent, the manager, the administrator and the third-party applications.

The overall use-case diagram (figure 6.1) presents all the actors, the generalisation of the supervising agent from the beekeeping agent, as well as all the dependencies between cases (include and extend relationships).

6.1 UC1: Plan and approve a visit

- **Actor**: supervising agent.
- **Preconditions**: the hive exists and the agent is authenticated.
- **Main scenario**: the agent proposes a visit date, the supervisor reviews the schedule then approves, the system records the approved status.
- **Alternatives**: the supervisor refuses and requests a new date.
- **Postcondition**: the visit is scheduled and visible in the calendar.

6.2 UC2: Carry out a visit and fill in the report

- **Actor**: beekeeping agent responsible for the hive.
- **Preconditions**: a visit is planned and approved.
- **Main scenario**: the agent enters date, time, duration, reason, findings, planned and carried-out actions, recommendations, then assesses the population, the health status and the productivity from 1 to 3.
- **Alternatives**: offline entry then deferred synchronisation, adding photos.
- **Postcondition**: the report is saved and the calendar cell switches to the carried-out status.

6.3 UC3: View the production dashboard

- **Actor**: farmer or owner.
- **Preconditions**: production measurements exist.
- **Main scenario**: the farmer filters by farm, site and period, the system displays the hives whose production exceeds the threshold and proposes harvest or extension.

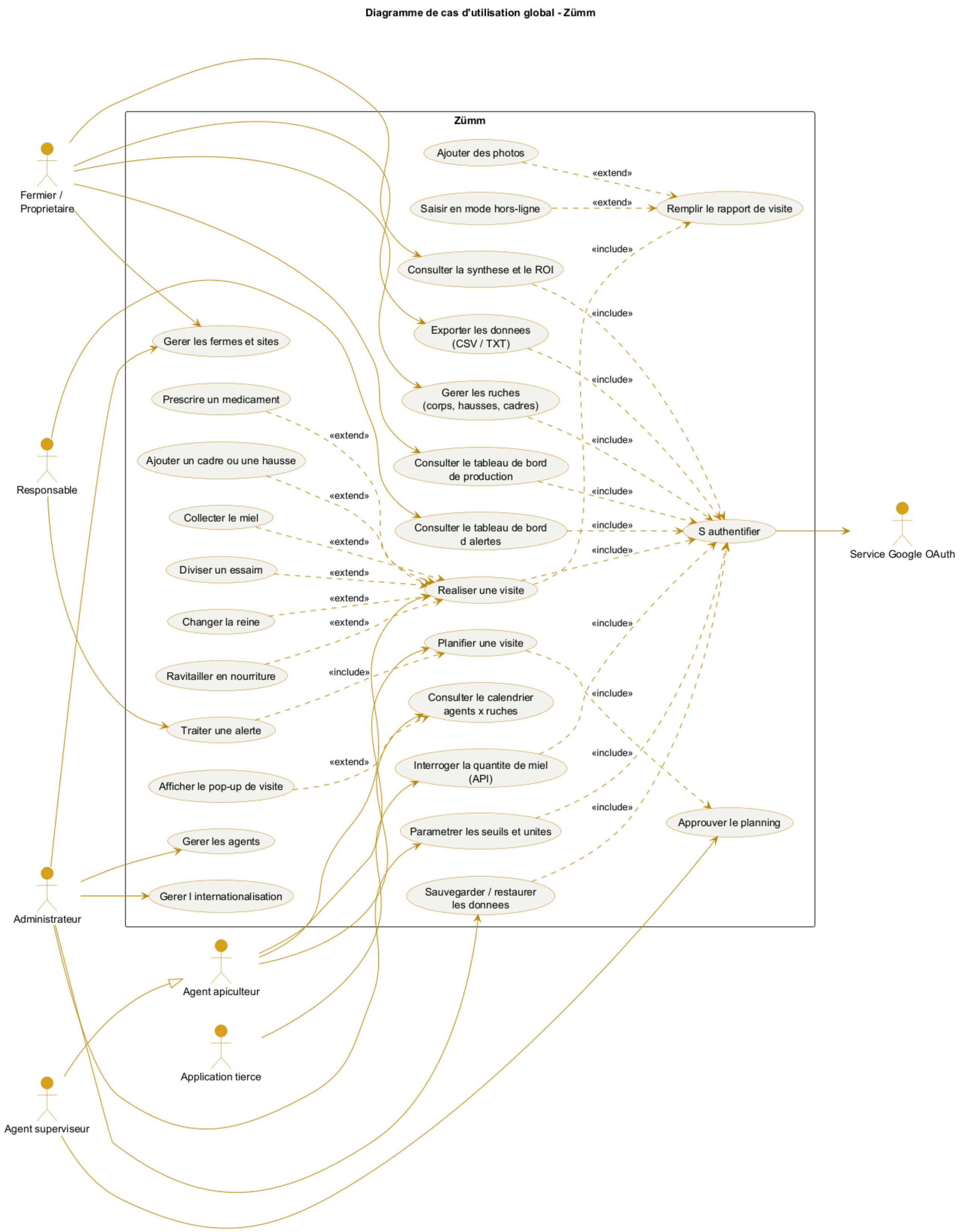

Figure 6.1. Overall use-case diagram with actors and dependencies (include and extend).

- **Postcondition**: the farmer decides on an operation.

6.4 UC4: Handle a health alert

- **Actor**: manager.
- **Preconditions**: an abnormal drop has been detected.
- **Main scenario**: the system flags the hive concerned, the manager plans an imminent inspection.
- **Postcondition**: an inspection visit is scheduled.

6.5 UC5: Query the honey quantity via the API

- **Actor**: third-party application (Excel or other).
- **Preconditions**: the application has secure access to the web service.
- **Main scenario**: the application calls `getZummHoneyActualQuantity(zummID)`, the service computes and returns the honey quantity.
- **Postcondition**: the value is returned in the XML web-service format.

CHAPTER 7

Design (UML models)

The design file comprises the following UML diagrams. Each one is described by its expected content in order to guide the modelling.

Diagram	Expected content
Use case	Actors (farmer, agent, supervisor, manager, administrator, third-party application) and main cases (see figure 6.1).
Class	Entities of the data dictionary and their relationships (Farmer 1 to n Farm, Farm 1 to n Site, Site 1 to n Hive, Hive 1 to n Body or super, Body or super 1 to n Frame) and service methods.
Object	Snapshot of a site containing three populated hives with their frames.
Sequence	Course of carrying out a visit (Agent, React client, REST controller, business service, repository, database).
Activity	Process of planning, approving, executing and closing a visit.
State machine	Life cycle of a hive (created, populated, active, splitting, harvesting, closed).
Interaction or collaboration	Same scenario as the sequence, in the form of a collaboration between objects.
Package	Presentation, control, business, data-access, internationalisation, configuration and web-services layers.
Component	Application components (web application, OAuth service, API service, i18n module, configuration module, database).
Deployment	Distribution across the browser, the reverse proxy, the application container, the DBMS, the Keycloak identity provider and the sensors.

7.1 Class diagram

The class diagram, presented in horizontal alignment, describes the business entities, their attributes, the service method `getZummHoneyActualQuantity` and the relationships with their cardinalities.

7.2 Layered view (package)

7.3 Deployment view

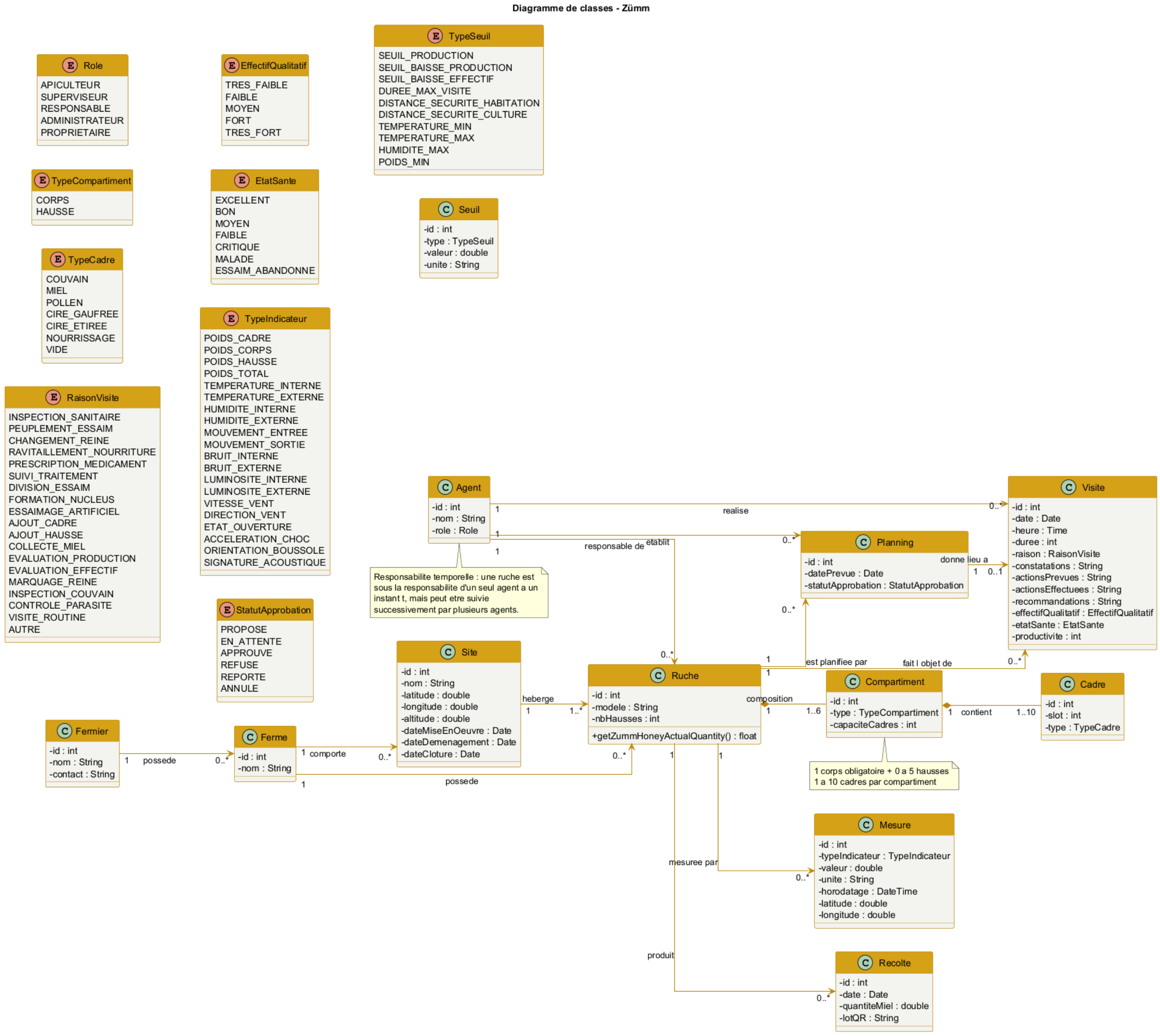

Figure 7.1. Class diagram of the Zümm system (horizontal alignment).

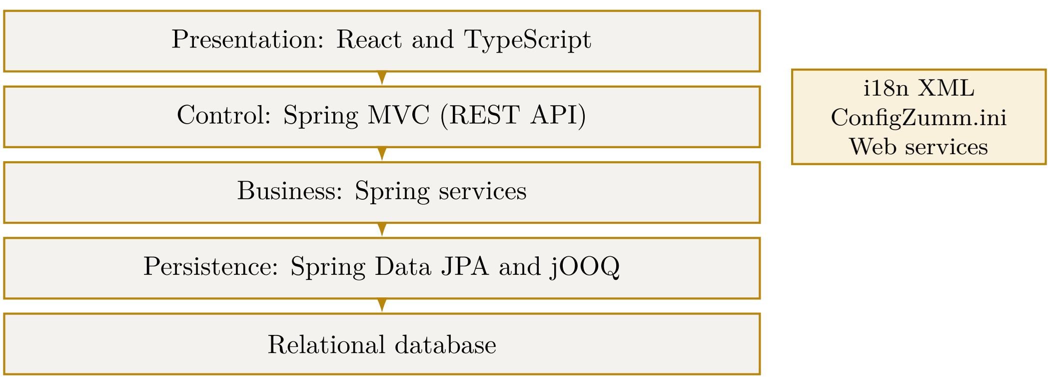

Figure 7.2. Multi-tier architecture in loosely coupled layers.

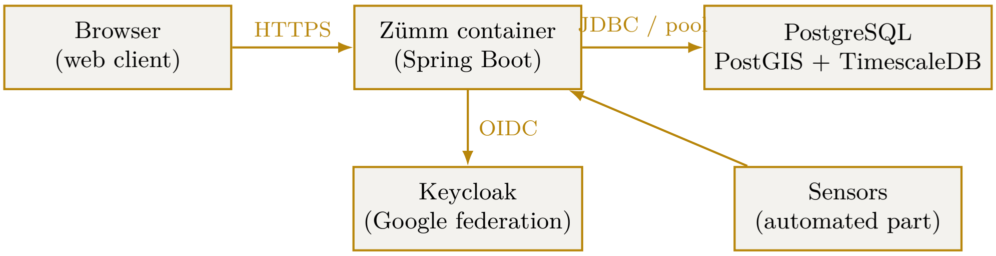

Figure 7.3. Deployment diagram of the system.

7.4 Sequence diagram (UC2)

The sequence diagram below details use case UC2, carry out a visit and fill in the report. It shows the passage through the layers (agent, React client, REST controller, business service, database) as well as the online and offline cases with deferred synchronisation.

7.5 Authentication sequence (Google OAuth)

The authentication flow follows the *Authorization Code* flow of OAuth 2.0. The browser never carries the client secret: the exchange of the authorization code for the access token is done **server to server**, over an SSL channel authenticated by an X.509 certificate. The application session is then opened by means of a secure cookie (`HttpOnly`, `Secure`), and the profile returned by Google is associated with an existing `Agent` (or creates one pending authorisation). In the demonstration, a local authentication fallback is provided in case the provider is unavailable.

7.6 Object diagram

The object diagram presents a snapshot of a site hosting three hives, with the detail of the composition of a hive into body, super and frames.

7.7 Activity diagram

The activity diagram models the process of planning, approving and carrying out a visit, with the online and offline variants.

7.8 State-machine diagram

The state-machine diagram describes the life cycle of a hive, from its creation to its closure.

7.9 Collaboration diagram

The collaboration diagram takes up the UC2 scenario in the form of numbered messages exchanged between the objects.

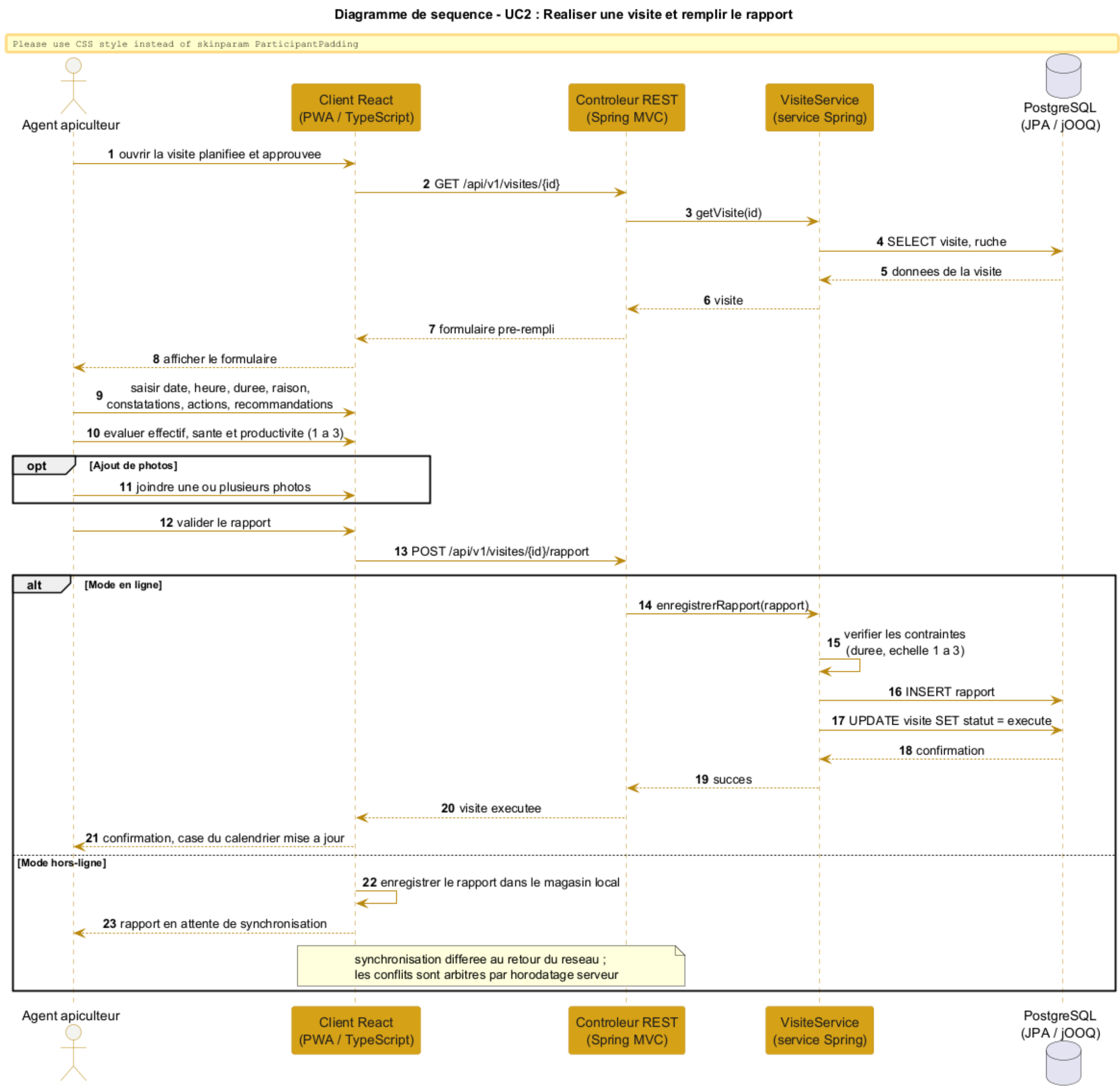

Figure 7.4. Sequence diagram of use case UC2 (carry out a visit).

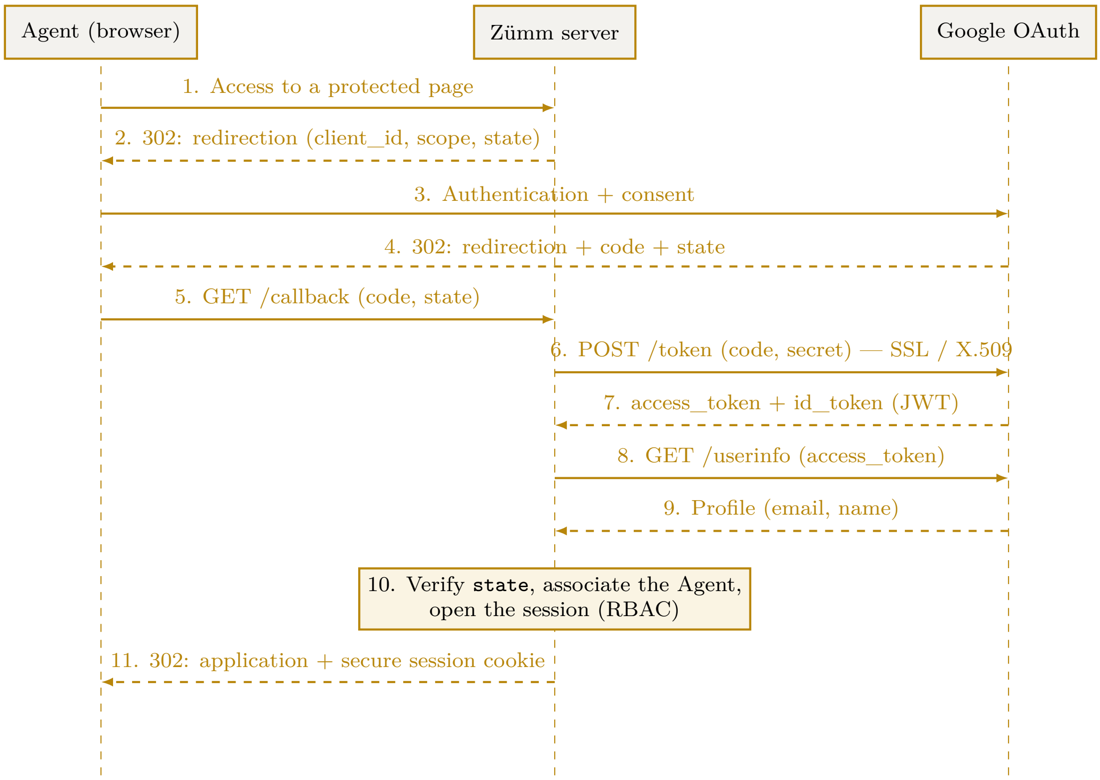

Figure 7.5. OAuth 2.0 authentication sequence (Authorization Code) with Google.

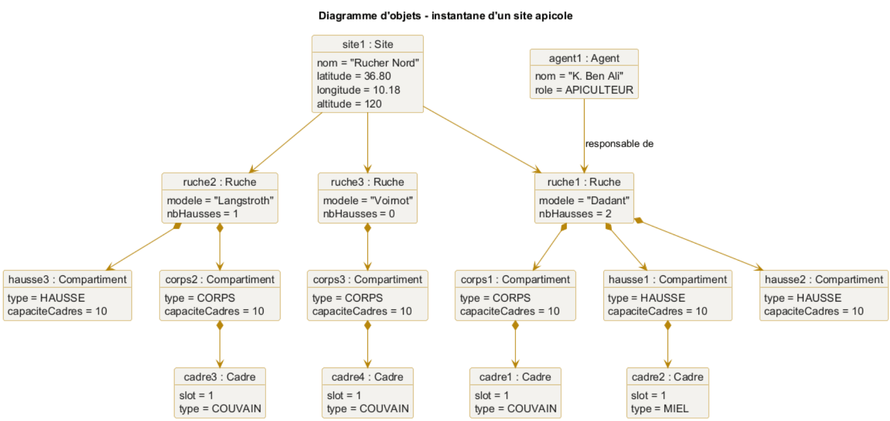

Figure 7.6. Object diagram, snapshot of an apiary site.

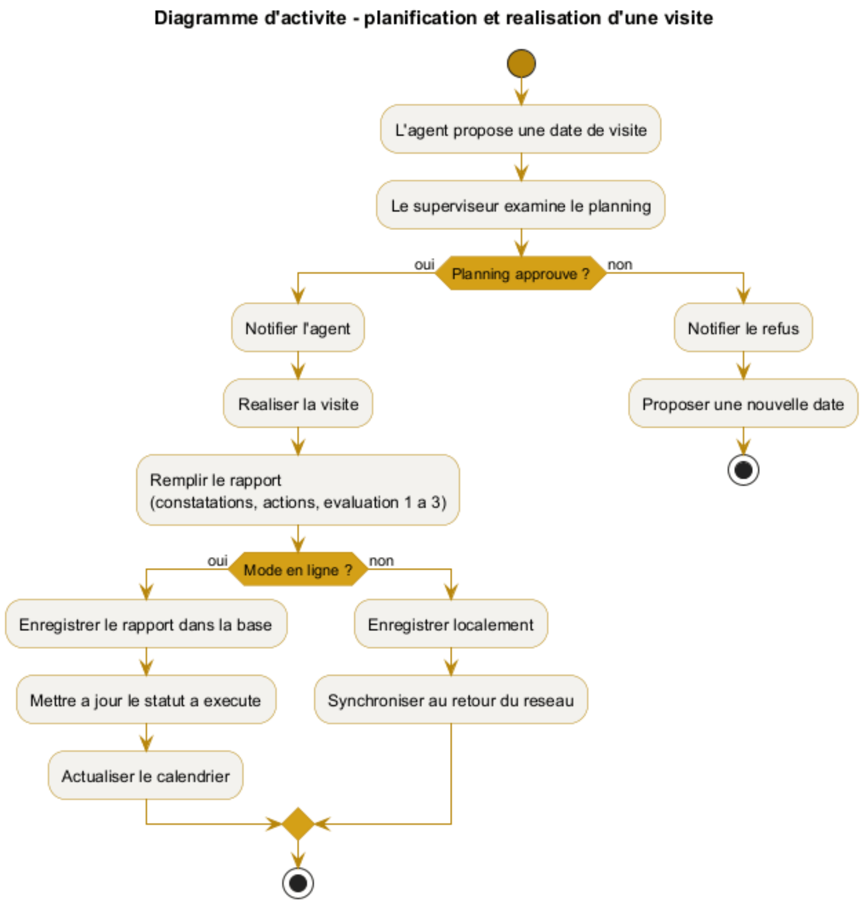

Figure 7.7. Activity diagram of the visit process.

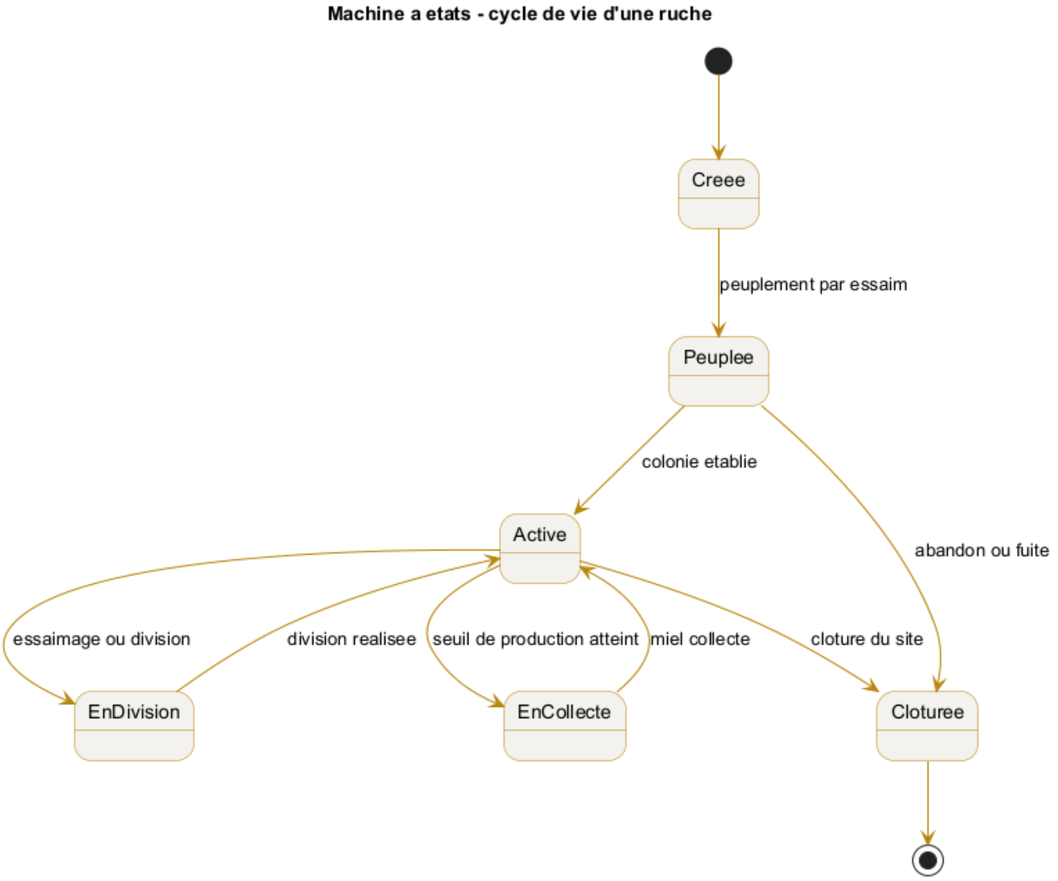

Figure 7.8. State machine of the life cycle of a hive.

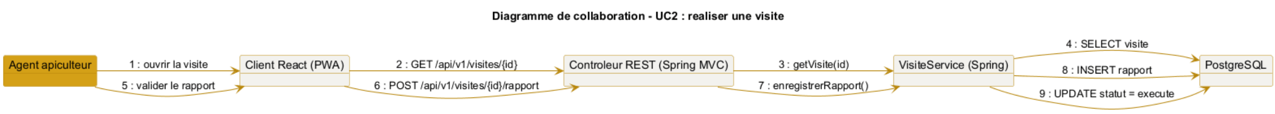

Figure 7.9. Collaboration diagram of use case UC2.

7.10 Component diagram

The component diagram shows the organisation of the application components, their interfaces and their dependencies, including the service interface exposing the honey quantity.

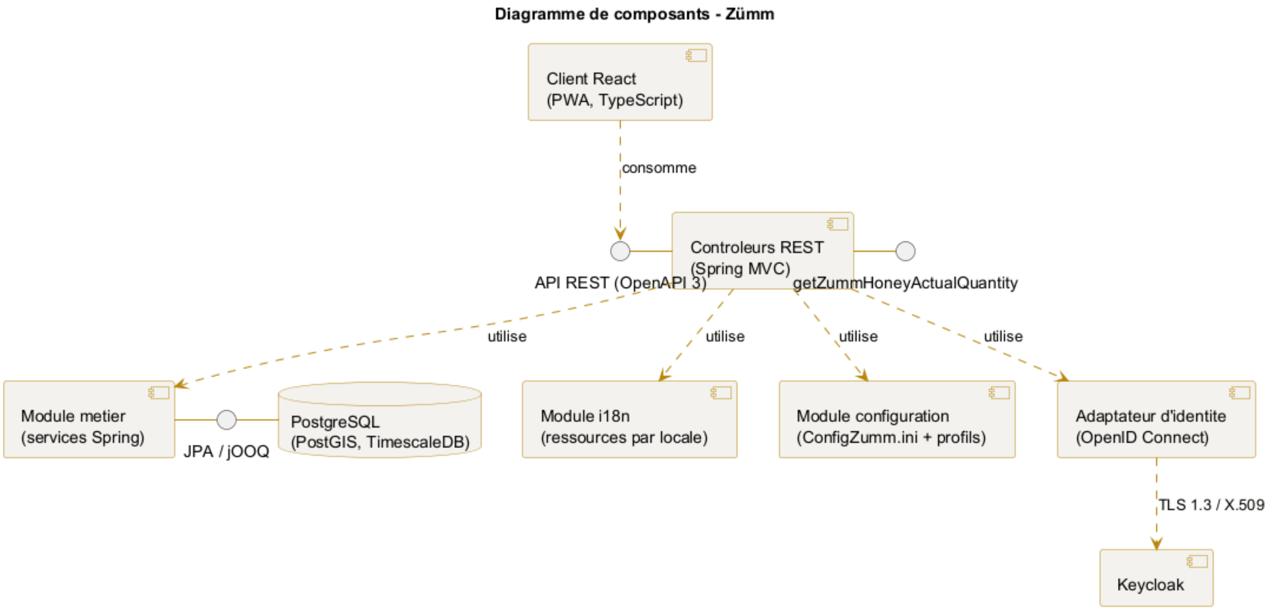

Figure 7.10. Component diagram of the Zümm system.

CHAPTER 8

Constraints and technical requirements

8.1 Architecture

A **scalable, evolvable, flexible, multi-tier and loosely coupled** architecture, relying on **Spring Boot 3** (Java 17 LTS) as the main infrastructure, with a strict separation between the service API and the presentation client.

8.2 Technical requirements adopted

Requirement	Description
Layered architecture	Controller / service / persistence separation, dependency inversion provided by the Spring container.
REST API	Spring MVC controllers exposing a versioned API, described by an **OpenAPI 3** contract.
Presentation	**React** client in **TypeScript**, decoupled from the server and consuming the REST API.
Business services	Spring components (`@Service`) encapsulating the business rules.
Persistence	**Spring Data JPA** for CRUD, **jOOQ** for analytical and geospatial queries.
Internationalisation	UI texts externalised in per-locale resource files, open to offline translation with no language limit, FR, EN and AR at minimum in the demonstration.
Technical configuration	Parameterisation via Spring profiles (`application.yml`) and environment variables: data sources, endpoints, secrets.
Business configuration	Thresholds, units and active modules in the text file `ConfigZumm.ini`, adjustable by the operator without recompilation or redeployment.
Inter-component security	**X.509** certificates and **TLS 1.3** protocol.
User authentication	**OpenID Connect** via **Keycloak**, with Google identity federation and **JWT** tokens.
Third-party API	REST endpoint exposing the `getZummHoneyActualQuantity(int zummID)` operation to retrieve the honey quantity of a hive, published in the OpenAPI contract.

Requirement	Description
Front-end	JavaScript / TypeScript and free libraries, respecting the imposed usability.

Identifier stability: the change of technical foundation alters neither the business domain nor the public identifiers. The `getZummHoneyActualQuantity` operation keeps its name and signature, whatever transport technology is chosen to expose it.

8.3 Non-functional requirements

- **Usability, user-friendliness and simplicity** of use, interface consistency and quality of the graphic design (of paramount importance).
- **Scalability**: openness to adding languages, units and indicators.
- **Interoperability**: API queryable by third-party applications from a published contract.
- **Security**: encryption of exchanges and strong authentication delegated to an identity provider.
- **Design quality**: compliance with the **SOLID** principles and use of **design patterns** to guarantee loose coupling and scalability, as detailed in appendix E.

Design principles adopted: the architecture applies the five **SOLID** principles (single responsibility, open/closed, Liskov substitution, interface segregation, dependency inversion) and a set of **design patterns** (Composite, State, Strategy, Observer, Command, Factory Method, Builder, Adapter, Facade, Singleton, Proxy). Their application to the system, justified "before / after", is detailed in appendix E.

8.4 Design questions to cover

The design must explicitly answer the following questions:

1. What are the **inputs** (list, data dictionary and types) and **outputs** (list, dictionary, types, formulas and calculation queries) of the system?
2. What does the solution consist of?
3. Who are the typical users and their access rights to the features?
4. How does the system carry out the user operations?
5. How is the system packaged and deployed?
6. How do the elements interact and in what order / chronology?
7. Why, how and where was each technology used?

8.5 Justification of the technology choices

The following table answers the why, the how and the where of each adopted technology. The **concrete implementations** of this foundation (runtime, DBMS, front-end libraries, API style) and the **justified comparison** of the evaluated alternatives are detailed in appendix D.

Technology	Why	Where
Spring Boot 3	Market-standard application foundation, injection container and autoconfiguration	General organisation of the application
Spring MVC	Process the HTTP requests and orchestrate the processing	Control layer
React + TypeScript	Produce a rich, typed and testable interface	Presentation layer
Spring Data JPA	Access the data without hand-writing the CRUD	Persistence layer
jOOQ	Express in typed SQL the analytical and PostGIS queries that JPA renders poorly	Dashboards and geoprocessing
OpenAPI 3	Publish a machine-readable contract and generate third-party clients	Interoperability interface
i18n resources	Externalise the texts for offline translation	Internationalisation module
Spring profiles	Parameterise the infrastructure per environment without rebuilding the image	Technical configuration
ConfigZumm.ini	Let the operator adjust business thresholds without technical intervention	Business configuration
Keycloak / OIDC	Authenticate and authorise without managing passwords or re-coding RBAC	Access to the system
X.509 and TLS 1.3	Encrypt and authenticate the exchanges	Inter-component communications

8.6 Packaging and deployment

- **Packaging**: the server application is packaged as an **executable Java archive** (self-contained JAR, embedded Tomcat server), then as a **container image** including the internationalisation resource files. The React client is built into static files served by the reverse proxy.
- **Runtime**: deployment of orchestrated containers, with no external application server to administer.
- **Database**: **PostgreSQL** extended by **PostGIS** (geolocation) and **TimescaleDB** (time series of measurements).
- **Security**: **TLS** termination at the reverse proxy with X.509 certificates, no application or management port published directly, configuration of the Keycloak identity realm.
- **Configuration**: the infrastructure is set through Spring profiles and environment variables; the business thresholds, units and active modules remain adjusted in `ConfigZumm.ini`, mounted as a volume and reloadable without redeploying the image.

The physical distribution is illustrated by the deployment diagram (figure 7.3).

CHAPTER 9

Budget envelope and resources

9.1 Framework

The project falls within an **academic (mini-project)** framework. The budget envelope therefore mainly translates into the **human, software and hardware resources** mobilised, with no direct financial cost.

9.2 Mobilised resources

Resource	Detail
Human	Student project team (design, development, testing, documentation).
Software	Containerised runtime environment, relational DBMS, IDE, UML tools, compilation chain.
Hardware	Development workstations, simulated sensors for the indicators part.
External	Google OAuth account, X.509 certificates (self-signed in the demonstration).

Major academic constraint: any use of code generators (ChatGPT, DeepSeek and derivatives) is prohibited and will be considered fraud in the examination.

9.3 Economic valuation: industrialisation hypothesis

The previous section describes the *actual* cost of the project, nil in direct spending. As a valuation exercise, this section estimates what the same product would cost **if delivered by a software services company**. The amounts are **working hypotheses**, not committed expenditure.

9.3.1 Reference workload

The workload plan in the schedule chapter retains **304 story points** across 8 sprints. At 0.79 person-days per point, the workload amounts to ≈ 240 person-days; at 1 person-day per point — a prudent assumption for a team new to the stack — it reaches ≈ 304 person-days. This range forms the basis of the costing.

9.3.2 Breakdown of a reference envelope

For an envelope of **USD 200,000**, the following breakdown reflects market practice. Development is not the majority share: equating it with the total cost is a classic costing mistake.

Item	%	USD	Rationale
Development	45	90,000	Back-end, front-end, integration
Project management / analysis	12	24,000	The most frequently underestimated item
Quality and testing	10	20,000	Coverage $\geq 70\,\%$, integration and load testing
UX/UI design and accessibility	8	16,000	Reduced cost: the design system already exists
Infrastructure and tooling (year 1)	6	12,000	Hosting, backups, monitoring
External security audit	5	10,000	Sensitive geolocation data
Contingency reserve	14	28,000	Without a reserve, an envelope of this size overruns

9.3.3 Sensitivity to the daily rate

The dominant factor is geographic rather than technical. At an average rate of USD 600–900 per person-day, the envelope funds $\approx$ 220–280 person-days: it barely covers the reference workload, **with no margin**. At a rate of USD 250–400, it funds $\approx$ 500–700 person-days and allows a team of five to six people together with a sound contingency reserve.

9.3.4 Adjustable scope

Should the envelope tighten, the following order of removal preserves the core of the product: first the advanced features (annex: queen tracking, batch QR code), then adaptive anomaly detection — whose calibration without real historical data is an identified risk — and finally live sensor ingestion, the data model being sufficient for a first version. The business entities, visits, authentication and access control, offline mode and dashboards remain incompressible.

9.3.5 Total cost of ownership

Build cost represents only part of the product's lifetime cost. Corrective and evolutionary maintenance is usually costed at **15–20 % per year** of the build envelope, i.e. USD 30,000–40,000 annually. Over three years, the total cost of ownership therefore approaches **USD 300,000**. To this must be added the items regularly omitted from an initial costing: migration of the operator's existing data, training and change management for field users, professionally reviewed translation for the three languages, and compliance work relating to personal data.

Scope of this section: this is an educational *valuation* intended to situate the project's effort within market orders of magnitude. It commits no expenditure and does not alter the academic framework defined at the beginning of this chapter.

CHAPTER 10

Schedule and deliverables

10.1 Expected deliverables

- Functional application (Java server source code + front-end).
- Test plan and execution results.
- Design file (UML diagrams).
- Project report, poster and oral presentation.

10.2 Acceptance criteria

Acceptance is pronounced when **all** the following criteria are satisfied and jointly verified. Every criterion is measurable: a criterion that cannot be measured is not an acceptance criterion.

Area	Criterion	Means of proof
Functional	The use cases of chapter 6 are implemented and demonstrated on real data.	Acceptance report
Code quality	Business-layer test coverage ≥ 70 %, no untreated critical or high vulnerability.	JaCoCo report, dependency scan
Security	External penetration test carried out; no critical or high vulnerability open at delivery.	Pentest report and remediation plan
Data protection	Geolocation coordinates inaccessible outside an authorised role, absent from default exports and from logs.	Authorisation tests, log review
Performance	p95 < 500 ms and error rate < 1 % under the nominal load defined in appendix I.	k6 load-test report
Availability	The service (SLO), recovery (RTO) and maximum loss (RPO) objectives of appendix I are met.	Monitoring, restoration test
Backup	A full restoration has been carried out successfully under real conditions.	Restoration test report

Area	Criterion	Means of proof
Accessibility and usability	Interface operable by keyboard, contrasts compliant with the design system, journey validated by a business user.	Accessibility audit, user acceptance
Internationalisation	The three languages (FR, EN, AR) are complete, Arabic RTL rendering verified on every screen.	Linguistic review
Operability	Runbook delivered, knowledge transfer completed, system redeployable from the repository.	Transfer session, dry-run redeployment
Reversibility	Data exportable in an open, documented format; source code and documentation handed over.	Demonstration export

Reservations. A minor reservation does not prevent acceptance but is subject to an agreed clearance deadline. A **blocking** reservation — any critical vulnerability, any restoration failure, any data leak between holdings — suspends acceptance until correction and re-verification.

10.3 UML diagrams to produce

Diagrams of: use case, classes, objects, sequence, activity, state machine, interaction / collaboration, package, component and deployment. The expected content of each is detailed in chapter 7 on the design.

10.4 Provisional schedule

The schedule is expressed in relative weeks, with the intermediate deliverables associated with each phase.

Phase	Weeks	Deliverable
Analysis and requirements specification	W1 to W2	Validated requirements specification
Design (UML)	W3 to W4	Design file
CRUD core development	W5 to W7	Model and entity management
Dashboards and alerts	W8 to W9	Calendar, alerts and summaries
Web services, security, i18n	W10 to W11	OAuth, API, X.509 and SSL, XML
Testing and validation	W12	Executed test plan
Documentation and presentation	W13	Report, poster and defence

CHAPTER 11

Test plan and validation

The test strategy covers the business functions, the management constraints and the technical requirements. Each test case specifies the target function, the preconditions, the steps and the expected result.

11.1 Test cases

ID	Function	Steps	Expected result
TC01	Hive CRUD	Create, read, modify, delete a hive	Compliant and persisted operations
TC02	Frame constraint	Add an 11th frame to a body	Rejected, maximum of 10 frames
TC03	Super constraint	Add a 6th super	Rejected, maximum of 5 supers
TC04	Base constraint	Remove the last frame of the body	Rejected, at least 1 frame
TC05	Visit schedule	Propose then approve a visit	Approved status recorded
TC06	Visit report	Fill in all fields and save	Report persisted, cell carried out
TC07	Production alert	Exceed the production threshold	Harvest alert displayed
TC08	Drop alert	Simulate a population drop	Inspection alert displayed
TC09	ROI calculation	Enter costs and production	ROI computed correctly
TC10	Honey API	Call getZummHoneyActualQuantity	Correct quantity returned
TC11	Internationalisation	Switch FR, EN then AR	Texts translated via the XML file
TC12	Configuration	Modify a threshold in ConfigZumm.ini	New threshold taken into account
TC13	Authentication	Log in via Google OAuth	Access granted
TC14	Security	Verify SSL and X.509 certificate	Encrypted and validated exchanges

ID	Function	Steps	Expected result
TC15	Export	Export the records	CSV and TXT files generated
TC16	Offline mode	Enter without a network then reconnect	Successful synchronisation

11.2 Traceability matrix

The matrix links each requirement to its use case, to the classes and tables that carry it, to the screen (mockup) concerned and to the test cases that validate it. It ensures end-to-end traceability, from the need to the verification.

Requirement	UC	Classes / Tables	Screen	Tests
Entity management and constraints	—	Hive, Compartment, Frame	Hive form (fig. F.5)	TC01–TC04
Planning and approval	UC1	Schedule, Agent	Calendar (fig. F.2)	TC05
Visit execution and report	UC2	Visit, Photo	Report (fig. F.4)	TC06
Production dashboard	UC3	Harvest, Measurement, Threshold	Prod. dashboard (fig. F.3)	TC07, TC09
Health / drop alert	UC4	Measurement, Threshold	Calendar	TC08
API web service (honey)	UC5	Hive, Harvest	—	TC10
Internationalisation	—	(i18n XML)	All	TC11
Threshold configuration	—	Threshold	Prod. dashboard	TC12
OAuth authentication	—	Agent	Login (fig. 7.5)	TC13
Exchange security	—	(SSL / X.509)	—	TC14
Portability and export	—	(CSV/TXT export)	All	TC15
Offline mode	UC2	Visit (local queue)	Report	TC16

CHAPTER 12

Beekeeping domain reference and good practices

This chapter specifies the domain knowledge that underpins the management rules, the configurable thresholds and the system indicators. It constitutes the functional basis on which the dashboards and the alerts rely.

12.1 Honey production factors

A colony's production depends directly on the weather, pollen availability and nectar flow. When the nectar flow is high, the queen lays more, the population peaks and the yield reaches its maximum. In a period of food shortage, laying and population decrease. The system must therefore correlate the measured production with periods and seasons, which justifies the filtering of the dashboards by month, week, season and year.

12.2 Location and configuration of a site

The choice of a site's location determines productivity. The following reference values can feed the fields and the input checks of the site-management module.

Criterion	Reference value
Proximity to resources	Nectar and pollen sources about 1 km away.
Access to water	Accessible drinking water, several litres consumed per day and per colony.
Distance from dwellings	100 m in a forest area, 200 m in a shrubland area, 300 m in an open area.
Distance from treated crops	At least 3 to 4 km from conventional crops using pesticides.
Elevation and inclination	Hives raised about 1.5 m off the ground and slightly tilted to drain moisture.
Protection	Shelter from intense sun, wind and floods.

Figure 12.1. Example of a hive installed on a site: elevation on legs and landing board.

12.3 Hive types and reference yield

Type of hive	Characteristics and yield
Traditional fixed-comb hive	6 to 10 kg of comb honey per season, not recommended in organic beekeeping.
Top-bar hive	Standard width of 32 to 35 cm, independent inspection of the combs.
Frame hive (Langstroth, East-African)	Honey combs reusable for several seasons after harvest.

These orders of magnitude serve as a basis for setting the production thresholds and estimating the return on investment per hive.

12.4 Classification of hive models

Two large families of hives are distinguished, whose modelling must remain open to cover local and international uses.

Family	Representative models
Frame hives (vertical)	Dadant, Langstroth, Voirnot, Layens, Aumônière, National (GB), WBC, Tonelli.
Traditional frameless hives	Warré, Kenyan TBH, Tanzanian TBH, log, straw (skep).

Family	Representative models
Other regional models	Lucitana, Oksman, Smith, CDB, Duvauchelle, Jarry, Congres, Zanders, Slovenian models.

In France, the most common models are the Dadant, Langstroth and Voirnot hives. All frame hives share a common composition, with dimensions precise to the millimetre, which reinforces the chosen data model.

Model anchor: the project's reference model is the Dadant hive, whose body holds up to 10 frames. This value grounds the maximum capacity of 10 frames per body or per super adopted in the management rules. The system nevertheless remains configurable to support other models.

12.4.1 Common composition of a frame hive

Element	Role
Floor / landing board	Base of the hive and landing zone for the bees.
Body (base)	Main brood compartment, holds the base frames.
Super	Extension level intended for honey harvesting.
Frame	Removable support of the wax combs, installed on an identified slot.
Queen excluder	Separates the body from the supers and confines the queen to the body.
Crown board and roof	Close and protect the hive from bad weather.
Feeder	Allows the food replenishment of the colony.

12.5 Colony inspection protocol

Inspections are preferably carried out on sunny days and focus on precise checkpoints that structure the visit report.

- brood development (eggs, larvae, capped cells)
- filling of the cells with honey and pollen
- presence of parasites or diseases
- collection of nectar, pollen and propolis

Operational constraint: a visit must not exceed 45 minutes. Beyond that, the colony's agitation increases and spreads to the neighbouring hives. The *duration* field of the visit report can be checked against this reference limit.

12.6 Swarming and colony splitting

An imminent swarming is signalled by the presence of queen cells on the edges of the combs. A few days before the emergence of a new queen, the old queen leaves the colony with half the workers. The

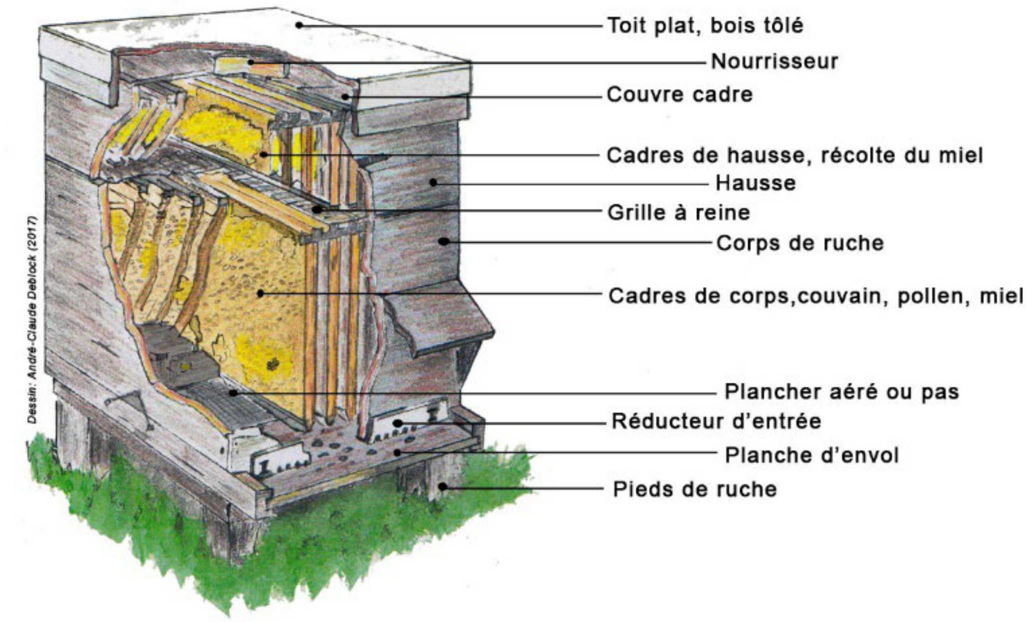

Figure 12.2. Cross-section of a frame hive and role of each element (roof, feeder, crown board, super, queen excluder, body, floor).

system must therefore make it possible to anticipate the split. Three reference methods are provided as operation reasons.

Method	Principle
Nucleus formation	Remove 1 to 2 brood combs with bees to a new hive.
Queen nucleus	Transfer the queen with a comb free of queen cells.
Artificial swarming	Move the mother hive and place a new hive at the old location with 2 to 3 combs of young brood.

12.7 Well-being indicators and causes of decline

A healthy colony presents a well-developed brood nest at the various stages, cells filled with resources, visible foraging activity and the absence of parasites or diseases. These criteria feed the qualitative assessment of the health status and the productivity level rated from 1 to 3 in the visit report.

The absconding or abandonment of a hive results from excessive disturbances or inadequate conditions (lack of food or water, extreme temperatures). Leaving a honey reserve to the colony during the harvest reduces this risk. These phenomena justify the alerts of sudden population or production drop and the associated configurable thresholds.

12.8 Impact on the configurable thresholds

System parameter	Business anchor
Honey-harvest threshold	Reference yield per hive type and per season.
Drop-alert threshold	Abnormal drop linked to absconding, disease or food shortage.
Maximum visit duration	Limit of 45 minutes per inspection.
Site safety distances	100, 200 or 300 m depending on the area, 3 to 4 km from treated crops.
Analysis periods	Correlation of production and season (nectar flow).

APPENDIX A

Appendix: Physical structure of a hive

For the record, a movable-frame hive is made up, from bottom to top, of the following elements: floor / landing board, body (base) holding the brood frames, queen excluder, one or more supers holding the honey-harvest frames, crown board and roof. This structure justifies the composition rules (mandatory body, up to 5 supers, 1 to 10 frames per element) adopted in the business model.

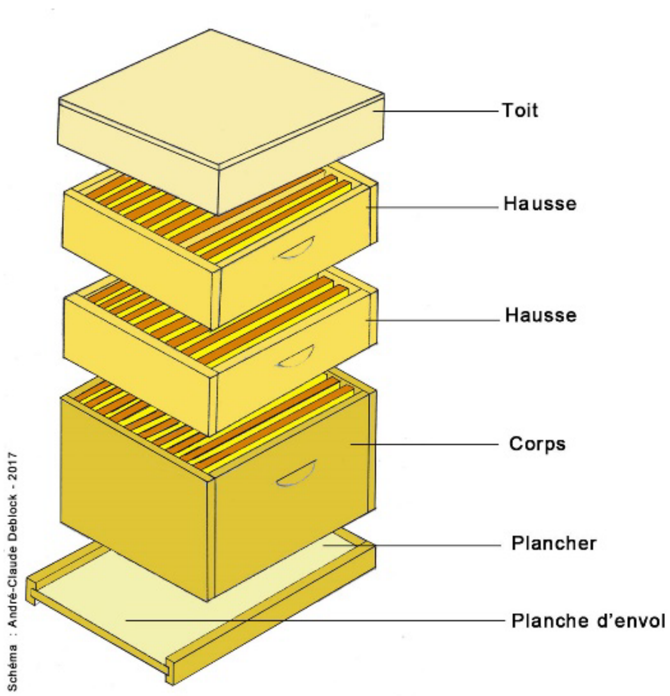

Figure A.1. Exploded view of a frame hive: roof, supers, body, floor and landing board.

APPENDIX B

Glossary

Term	Definition
Base or body	Base compartment of the hive holding the brood frames.
Super	Extension level added to the body for honey harvesting.
Frame	Removable support of the wax combs, installed on a slot.
Slot	Identified location of a frame in a body or a super.
Swarm	Colony of bees populating a hive.
Swarming	Departure of part of the colony with the queen to found a new colony.
Nucleus	Small colony formed to split a swarm or raise a queen.
Queen	The colony's single reproductive female.
Brood	All the eggs, larvae and pupae of the colony.
Foraging	Activity of collecting nectar and pollen by the bees.
Propolis	Resinous substance collected by the bees.
Queen excluder	Grid that confines the queen to the body and separates the supers.
Feeder	Device for the food replenishment of the colony.
Configurable threshold	Configurable reference value triggering an alert.
ROI	Return on investment, ratio between valued production and costs.
CRUD	Create, read, update and delete operations.
i18n	Internationalisation, adaptation to languages and unit systems.
Spring Boot	Java application framework: dependency injection, autoconfiguration and embedded server.
JPA	Standard mapping interface between Java objects and relational tables.
jOOQ	Library for typed SQL queries, checked at compile time.
OpenAPI	Contract describing a REST API, usable by humans and machines.
OAuth / OIDC	Protocols for delegated authorization and authentication.
X.509	Certificates ensuring authentication of the parties in an encrypted exchange.
TLS	Encryption protocol for network exchanges (transport layer), basis of HTTPS.
REST / JSON	Web-API style and text format for exchanging data (measurement ingestion).

Term	Definition
MQTT	Lightweight messaging protocol (publish/subscribe) suited to sensors and constrained networks.
JWT (token)	Signed identity-bearing token, issued by the OAuth provider.
RBAC	Role-based access control, applied server-side (see appendix F).
GDPR	General Data Protection Regulation. Governs personal data.
Retention	Duration for which a datum is kept before archiving, anonymisation or purge.
Encryption at rest	Encryption of stored data (sensitive columns) in the database.
Pseudonymisation	Dissociation of a person's identity from their data, reversible under control.
Idempotency	Property of an operation that can be replayed without creating a duplicate (measurement ingestion).
Hysteresis	Persistence of a breach over several measurements before raising an alert (debounce).

APPENDIX C

References and sources

This requirements specification draws on the following sources.

Elaboration method

- Manager-GO, *Élaborer un cahier des charges* (Drawing up a requirements specification).
  `https://www.manager-go.com/gestion-de-projet/dossiers-methodes/elaborer-un-cdc`

Software design principles

- Alex so yes, *The SOLID principles.*
  `https://alexsoyes.com/solid/`
- Refactoring Guru, *Design patterns catalogue.*
  `https://refactoring.guru`

Beekeeping domain reference

- Organic Africa, *Improving honey production.*
  `https://www.organic-africa.net/fr/agriculture-biologique/agriculture-biologique/animaux/apiculture/amelioration-de-la-production-de-miel.html`
- Au Bon Miel, *The hives.*
  `https://www.aubonmiel.com/les-ruches/`

Features of market solutions

- HiveTracks, beekeeping management application.
  `https://www.hivetracks.com/`
- Apiary Book, apiary management solution.
  `https://www.apiarybook.com/`
- BeePlus, beekeeping manager (App Store).

Connected monitoring (precision beekeeping)

- Arnia, acoustic monitoring and weighing of hives.
  `https://www.arnia.co/`

- BroodMinder, temperature, humidity and weight sensors.
  https://broodminder.com/
- BeeHero, IoT platform and artificial-intelligence analysis.
  https://www.beehero.io/

User feedback and limits of the solutions

- Beekeeping Diary, *Best Beekeeping Apps 2026 (comparison)*.
  https://beekeeping-diary.eu/blog/en/best-beekeeping-apps-2026/
- HiveBook, *Free HiveTracks Alternatives*.
  https://hivebook.app/blog/hivetracks-alternatives-free
- Beesource Forums, *Problems with Hive Tracks*.
  https://www.beesource.com/threads/problems-with-hive-tracks.249656/
- ApiTech World, *Smart Hive Monitoring, a Beekeeper's Reality Check (2026)*.
  https://apitechworld.com/smart-hive-monitoring-a-beekeepers-reality-check-2026-review/

APPENDIX D

Appendix: Technology stack and comparison of alternatives

The technical foundation of **Zümm** aims at **alignment with market practice**: documented REST API, decoupled client, federated identity, geospatial data and time series in a single DBMS. This appendix lists the technologies adopted layer by layer, then justifies each choice against its alternatives.

D.1 Adopted technology stack

The table below specifies, for each layer of the multi-tier architecture (figure 7.2), the concrete implementation adopted and its reference version. All the building blocks are **free and open source**, in line with the requirement of self-deployment without subscription.

Layer / need	Technology adopted	Role in Zümm
Platform	Spring Boot 3 on JDK 17 LTS	Application foundation: IoC container, auto-configuration, embedded Tomcat
Runtime	Container image (Docker)	Single deployable artefact, no application server to administer
Control	Spring MVC + Bean Validation	Request routing, input validation
Business	Spring `@Service` components	Business rules: ROI, thresholds, honey quantity
Data access	Spring Data JPA (CRUD) + jOOQ (typed SQL)	Entity persistence, analytical and geospatial queries
DBMS	PostgreSQL + PostGIS + TimescaleDB	Relational persistence, site geolocation, measurement series
Presentation	React 19 + TypeScript, Zümm tokens	Responsive UI, web / mobile parity, visual identity of the design system
Charts	Chart.js	Production, alert and summary dashboards
Mapping	MapLibre GL JS + OpenStreetMap tiles	Map of sites and foraging radii
i18n	Per-locale resource files	FR / EN / AR texts, Arabic RTL support

Layer / need	Technology adopted	Role in Zümm
Configuration	Spring profiles (technical) + `ConfigZumm.ini` via a dedicated `PropertySource` (business)	Infrastructure per environment; thresholds, units and modules without redeployment
Third-party API	REST + OpenAPI 3 contract	`getZummHoneyActualQuantity(int)`
Authentication	Keycloak (OpenID Connect) + Google federation	Delegated login, RBAC and JWT tokens
Exchange security	TLS 1.3 + X.509 certificates	Encryption and authentication between components
Offline	PWA (Service Worker) + synchronisation engine	Field data entry without network, conflict resolution when the network returns
Sensor ingestion	MQTT (EMQX broker)	Reception of measurements from connected hives
Observability	OpenTelemetry to Prometheus / Grafana	Traces, metrics and operational dashboards
Build & tests	Maven, JUnit 5, Mockito, Testcontainers	Compilation, unit and integration tests on a real database

Note: the business domain and the public identifiers are independent of the technical foundation. Class, table and operation names — including `getZummHoneyActualQuantity` — remain unchanged whatever technology carries them.

D.2 Justified comparison of the alternatives

For each open decision, the credible options were evaluated. The table gives the option adopted, cites the alternatives rejected and provides the justification with respect to the project requirements (self-deployment, free of charge, usability, geolocation, interoperability).

D.2.1 Application foundation

Option	Alternatives rejected	Justification of the choice
Spring Boot 3	Jakarta EE / WildFly, Quarkus, Micronaut, Node.js (NestJS)	**Most widespread ecosystem** in the Java market: abundant documentation, libraries and developer pool, hence controlled maintenance cost. Quarkus starts faster and consumes less, but its ecosystem is narrower. Jakarta EE imposes an application server to administer for no benefit at this scale.

D.2.2 Data access

Option	Alternatives rejected	Justification of the choice
JPA + jOOQ	Hand-written JDBC, MyBatis, JPA alone	**Mixed** approach: JPA removes the repetitive CRUD code over the fifteen or so domain entities, while jOOQ expresses in typed, compile-time-checked SQL the analytical queries and PostGIS calls that JPA makes unreadable. Hand-written JDBC would give the same result at a far higher development cost.

D.2.3 Database management system

Option	Alternatives rejected	Justification of the choice
PostgreSQL	MySQL / MariaDB, H2, Oracle XE	Native **geolocation** (PostGIS) for sites and foraging radii and **time series** (TimescaleDB) for sensor measurements, in a *single* instance: neither an additional specialised database nor synchronisation between two systems. Good support for `CHECK` constraints (frame/super rules). Free and without proprietary lock-in, unlike Oracle.

D.2.4 Third-party API style

Option	Alternatives rejected	Justification of the choice
REST + OpenAPI 3	SOAP (JAX-WS), GraphQL, gRPC	**De facto** standard for third-party integration: contract readable by humans and machines, automatic client generation in most languages, universal testing tooling. SOAP would only make sense to integrate with an existing system that imposes it. GraphQL addresses client querying needs that Zümm does not have.

D.2.5 User interface

Option	Alternatives rejected	Justification of the choice
React + TypeScript	JSP / Thymeleaf, Vue, Angular, Svelte	Complete **decoupling** of client and server, essential for offline operation: the same application serves as a PWA in the field. TypeScript secures the data contracts generated from OpenAPI. Server-side rendering (JSP, Thymeleaf) would rule out the disconnected mode.

D.2.6 Authentication and access control

Option	Alternatives rejected	Justification of the choice
Keycloak (OIDC)	Hand-wired Google OAuth, Auth0, Spring Security alone	Provides **natively** the project's three needs: Google federation, local fallback accounts and a role matrix. Re-coding that triplet would represent several weeks for a less secure result. Auth0 is equivalent but paid beyond a quota, which contradicts self-deployment.

D.2.7 Configuration externalisation

Option	Alternatives rejected	Justification of the choice
Mixed approach	Spring profiles alone, `ConfigZumm.ini` alone	The two configurations have neither the same audience nor the same life cycle. The **infrastructure** (data sources, endpoints, secrets) changes per environment and belongs to technical operations: Spring profiles and environment variables. The **business thresholds** (production, alerts, units) are set by the beekeepers themselves, during the season: they remain in `ConfigZumm.ini`, a readable text file, mounted as a volume and re-read at runtime by a dedicated `PropertySource`. Merging everything into `application.yml` would force the business user to go through a technician.

D.2.8 Mapping and foraging radii

Option	Alternatives rejected	Justification of the choice
MapLibre GL + OSM	Leaflet, Google Maps JS API, Mapbox	**Free and without a paid key** or billed quota, consistent with the self-deployment requirement. Vector tiles: crisp rendering at every zoom level and a style derivable from the design-system colours, which Leaflet's raster tiles do not allow.

D.2.9 Charting library

Option	Alternatives rejected	Justification of the choice
Chart.js	D3.js, ApexCharts, Highcharts	Gentle learning curve and direct rendering of the bar and line charts of the dashboards (production, ROI). D3 is more powerful but oversized. Highcharts is not free for commercial use.

D.2.10 Offline and mobile access

Option	Alternatives rejected	Justification of the choice
PWA + synchronisation engine	Native application (Android/iOS), Flutter, React Native, bare IndexedDB	**Web / mobile parity** with a single code base, field data entry without network then deferred synchronisation. A dedicated synchronisation engine brings a proven **conflict-resolution policy** — two agents editing the same visit offline is the nominal business case, not an edge case.

D.2.11 Integration testing

Option	Alternatives rejected	Justification of the choice
Testcontainers	Arquillian, in-memory H2 database	The tests run against a **real PostgreSQL** equipped with PostGIS and TimescaleDB, hence against the same extensions as in production — something an in-memory H2 database cannot simulate. Arquillian assumes an application server, which the adopted architecture no longer uses.

Consistency with the project constraints: all the building blocks adopted are free, documented and *justifiable* “why / how / where” (*Technologies* criterion of the assessment grid, deadlines chapter). None relies on a code generator: they are implemented by hand by the team, in compliance with the constraint of the examination.

APPENDIX E

Appendix – SOLID principles and design patterns

This appendix answers the question of the *how* of software quality. It documents the application of the five **SOLID** principles and a set of **design patterns** to the **Zümm** system, setting each decision against the initial design (*before*) and the refactored design (*after*). The objective is a **multi-tier, scalable and loosely coupled** architecture, compliant with the requirements of chapter 7.

> **Methodological sources:** the definitions adopted follow the presentation of the SOLID principles from *alexsoyes.com* and the pattern catalogue from *refactoring.guru*, adapted to the project's beekeeping domain.

E.1 The five SOLID principles

Principle	Before (initial design)	After (refactored)
S: Single responsibility	The `Ruche` entity carries the calculation method `getZummHoneyActualQuantity()`: it mixes state (data) and business logic. `Visite` groups data and report fields.	The calculation is extracted into `ServiceMielImpl` (via the Composite) and the report becomes `RapportVisite`. Each class now has only a single reason to change.
O: Open / closed	Alert detection and unit conversion would rely on `if / switch` on `typeSeuil` and `unite`: adding a rule or a unit requires modifying existing code.	`RegleAlerte` (Strategy) and `ConvertisseurUnite`: a new rule or unit = a new class, without touching the `GestionnaireAlertes`. Open to extension, closed to modification.
L: Liskov substitution	The use-case diagram generalises the supervising agent from the beekeeping agent. Transposed as is into class inheritance, a restrictive subtype would break substitutability.	`Corps` and `Hausse` inherit from `Compartiment` while fully respecting its contract (capacity 1..10, `getPoidsMiel`). The agent's role goes through the `Role` enumeration + permissions (composition) rather than a restrictive inheritance.

Principle	Before (initial design)	After (refactored)
I: Interface segregation	A single "catch-all" service (CRUD + visits + dashboards + honey API) would force the third-party application to depend on unused methods.	Targeted interfaces: `ServiceRequeteMiel` (the third-party API sees only `getZummHoneyActualQuantity`), `ServiceVisite`, `ServiceTableauDeBord`. Each client depends only on its contract.
D: Dependency inversion	The business layer calls SQL directly (concrete implementation): strong coupling to the database and to the result sets.	The business layer depends on abstractions `RucheRepository` / `VisiteRepository` (DAO pattern), implemented by Spring Data JPA and, for analytical queries, by jOOQ. Mock testing and implementation substitution facilitated (cf. appendix D).

E.2 Chosen design patterns

The following table lists the applied patterns, their category (creational, structural, behavioural) and the layer concerned, with the *before / after* justification.

Pattern	Before	After
Composite ***structural: business***	The honey calculation manually traverses the frame collections with duplicated summation code.	`ElementRuche.getPoidsMiel()` is resolved recursively (Ruche → Compartiment → Cadre): uniform processing of the tree.
State ***behavioural: business***	The hive's status would be tested by scattered `switch` (created, active, harvesting, closed, ...).	`EtatRuche` and its concrete states encapsulate the allowed transitions. Direct alignment with the state-machine diagram.
Strategy ***behavioural: business***	The alert rules, unit conversion and ROI would be hard-coded.	Families of interchangeable algorithms (`RegleAlerte`, `ConvertisseurUnite`), injectable and testable in isolation.
Observer ***behavioural: business***	The measurement code would call the dashboards directly (strong coupling).	`GestionnaireAlertes` notifies the subscribed `Observateur` (alerts dashboard, manager notification) when a threshold is crossed.
Command ***behavioural: control***	The field operations (harvest, split, requeening) are direct calls, not replayable offline.	Each operation becomes a `CommandeVisite`. The `FileCommandesHorsLigne` replays the commands on reconnection (offline mode + deferred synchronisation).
Factory Method ***creational: business***	The hives are created by scattered `new` with manual initialisation of the compartments.	`FabriqueRuche.creer(modele)` produces a preconfigured hive according to the model (Dadant, Langstroth...) with consistent body and capacities.

Pattern	Before	After
Builder ***creational: presentation***	The visit report, with its many optional fields, would require a telescoping constructor or scattered setters.	`ConstructeurRapport` builds `RapportVisite` fluently (findings, actions, photos, assessments).
Adapter ***structural: data***	The heterogeneous sensors (different formats and units) would be handled case by case.	`AdaptateurCapteur` standardises each source into a `Mesure` with configurable units (automated part).
Facade ***structural: control***	The controller orchestrates all the business services directly.	`FacadeApplication` offers a simplified entry point to the controllers on top of the subsystems.
Singleton ***creational: cross-cutting***	The `ConfigZumm.ini` file and the i18n labels would be re-read in several instances.	`Configuration` and `GestionnaireI18n` guarantee a single shared instance.
Proxy ***structural: web services***	Access to the API would directly expose the business service.	A `ProxySecurite` controls OAuth authentication and SSL / X.509 encryption before delegating to the service.

E.3 Refactored design view

Figure E.1 summarises the main patterns in a design class diagram: the Composite (hive composition and honey calculation), the State (life cycle), the segregated services (ISP), abstraction-based persistence (DIP / Repository), the Strategy + Observer pair (alerts) and the Factory / Builder / Command trio.

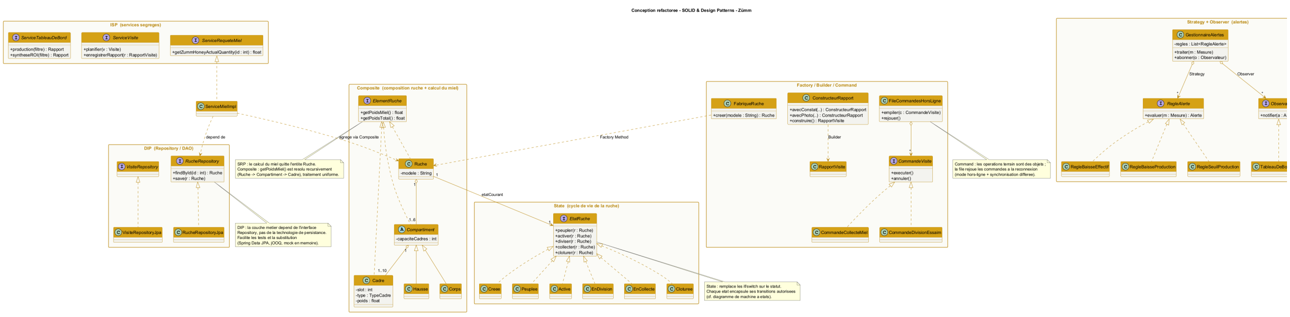

Figure E.1. Refactored class diagram: application of SOLID and the design patterns to the Zümm system.

> **Correspondence with the layers:** the `Repository` belong to the persistence layer (JPA / jOOQ), the `Service` and the domain (Composite, State, Strategy, Observer, Command) to the business layer (Spring services), the `Facade` and the `Proxy` to the control layer (Spring MVC), the report `Builder` to the presentation (React). The loosely coupled layered architecture of chapter 7 is thus reinforced, without being modified.

E.4 Dynamic illustration: Observer and Command patterns

Two sequence diagrams detail the behaviour of the system's most structuring patterns.

E.4.1 Observer (+ Strategy): triggering an alert

Figure E.2 shows how a measurement received by the `GestionnaireAlertes` (subject) is evaluated by an interchangeable rule (*Strategy*) then, in case of a breach, propagated to the subscribed observers (alerts dashboard, manager notification). Adding a rule or an observer requires no modification of the manager (open/closed principle).

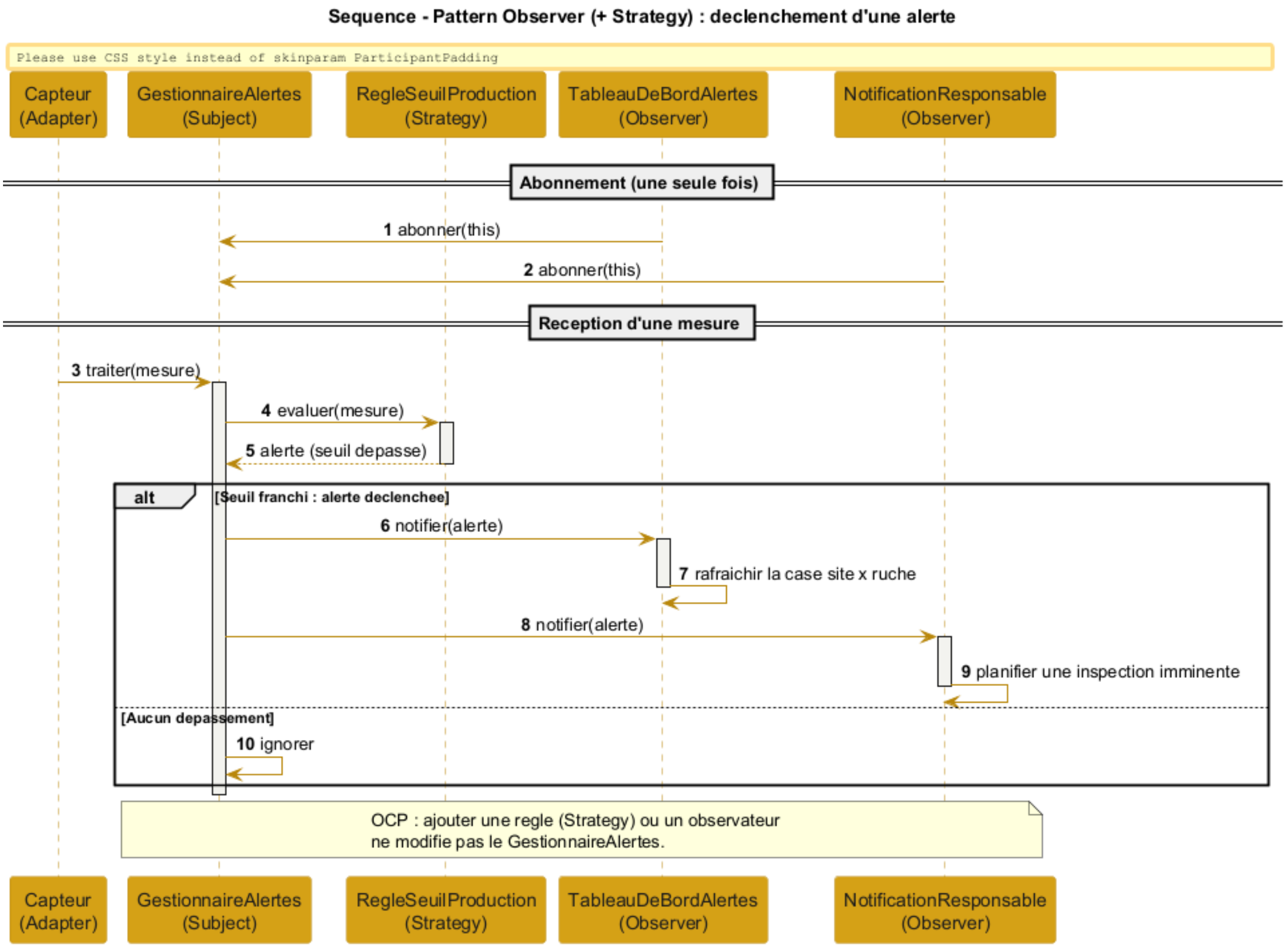

Figure E.2. Sequence of the Observer (+ Strategy) pattern: alert-triggering flow.

E.4.2 Command: offline mode and deferred synchronisation

Figure E.3 illustrates the entry of a field operation without a network: the operation is encapsulated in a `CommandeVisite` pushed onto the `FileCommandesHorsLigne`. When the network returns, the queue replays (`executer`) each command, or compensates it (`annuler`) in case of conflict, achieving the deferred synchronisation required by the offline mode.

E.4.3 State: hive life cycle

Figure E.4 shows the Ruche (context) delegating each operation to its current state. The state decides the transition (activation, harvesting, return to the active state) or refuses an operation forbidden in the current state, here a populating requested on an already active hive. The behaviour thus depends on the state without any `switch` on the status, consistently with the state-machine diagram (figure 7.8).

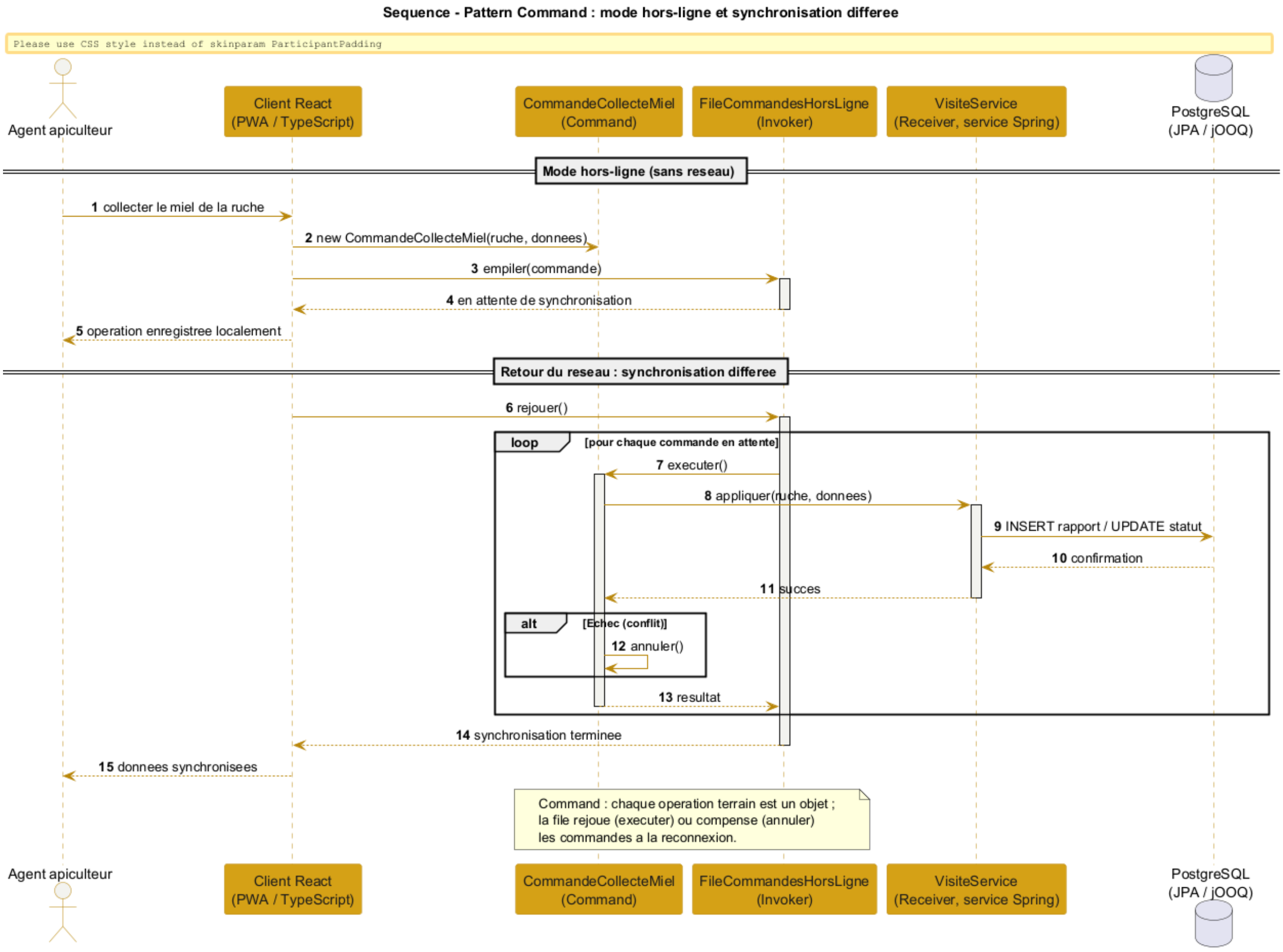

Figure E.3. Sequence of the Command pattern: offline mode and deferred synchronisation.

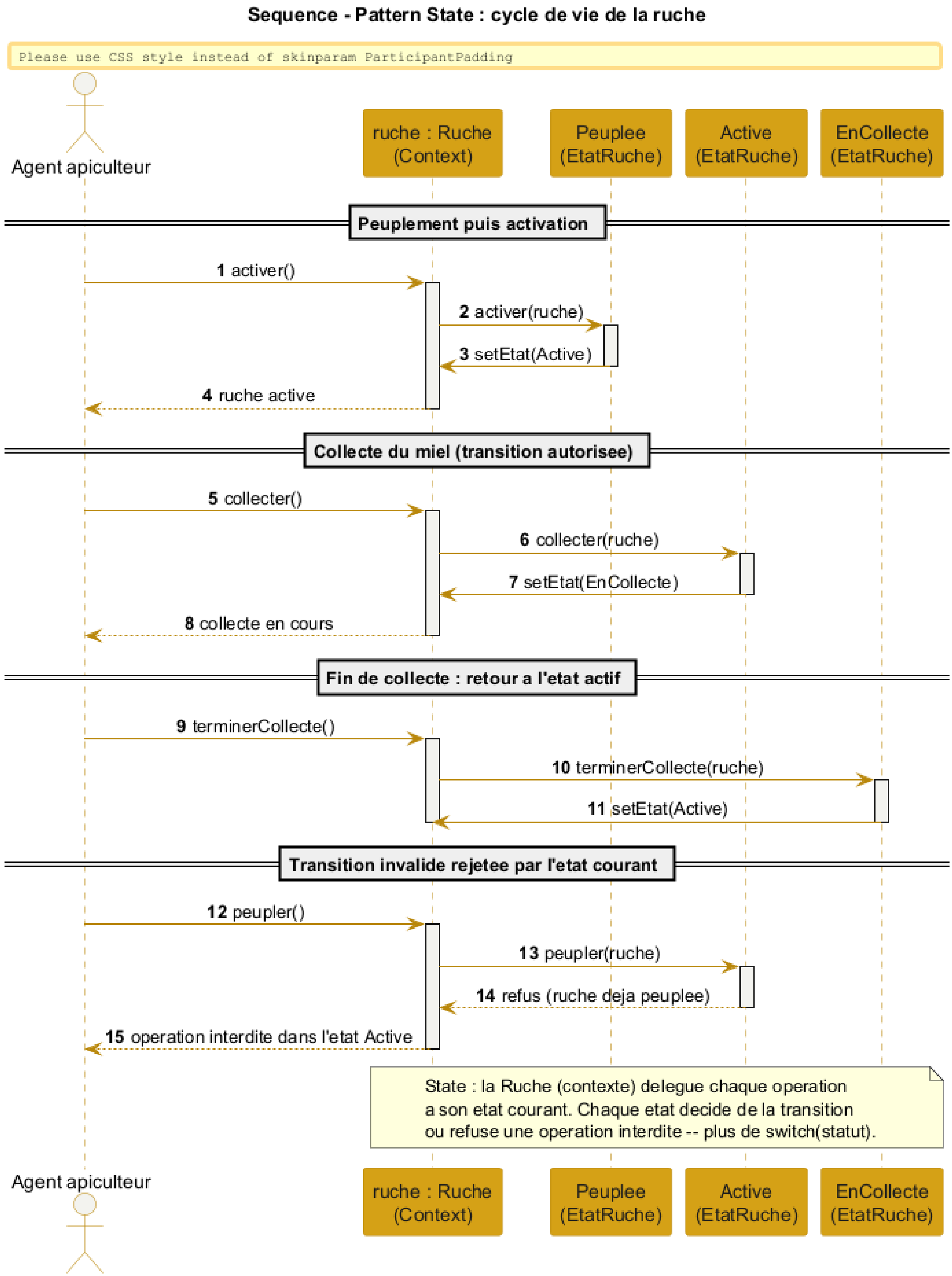

Figure E.4. Sequence of the State pattern: delegation of the transitions of the hive life cycle.

E.4.4 Composite: honey-quantity calculation

Figure E.5 details the recursive resolution of `getPoidsMiel()`: the client (`ServiceMielImpl`) queries the hive, which propagates the call to each compartment, then to each frame (leaf). Only frames of type `MIEL` contribute, and the tare is subtracted at the hive level. The client ignores the internal structure. This calculation implements the `getZummHoneyActualQuantity` method exposed by the web service.

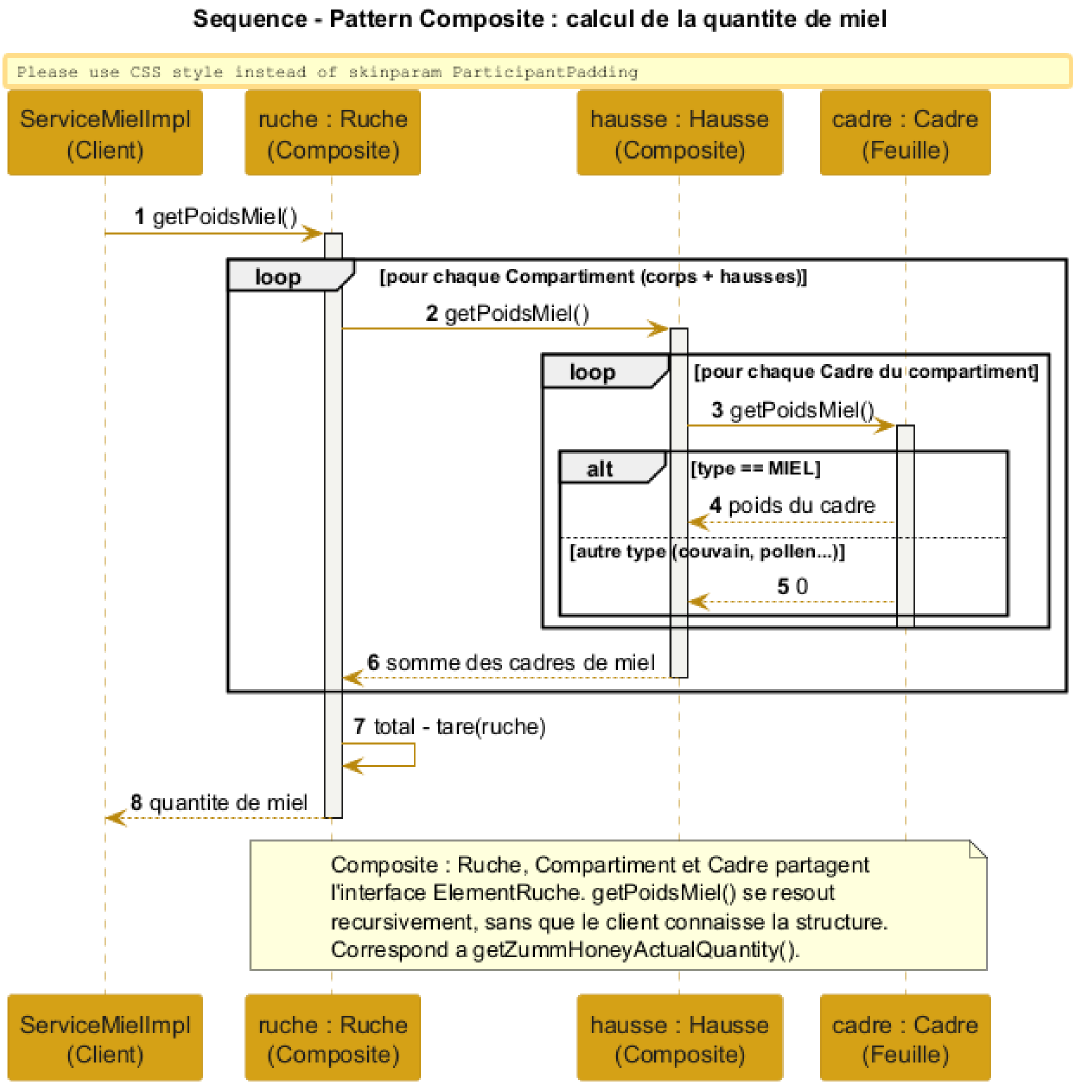

Figure E.5. Sequence of the Composite pattern: recursive calculation of a hive's honey quantity.

E.4.5 Strategy: conversion of heterogeneous units

Figure E.6 shows the normalisation of a measurement expressed in any unit: the `RegistreConvertisseurs` (context) selects the strategy corresponding to the unit (`ConvertisseurLbVersKg`), which brings the value back to the reference unit before comparison with the threshold. A new unit translates into a new strategy, without modifying the context.

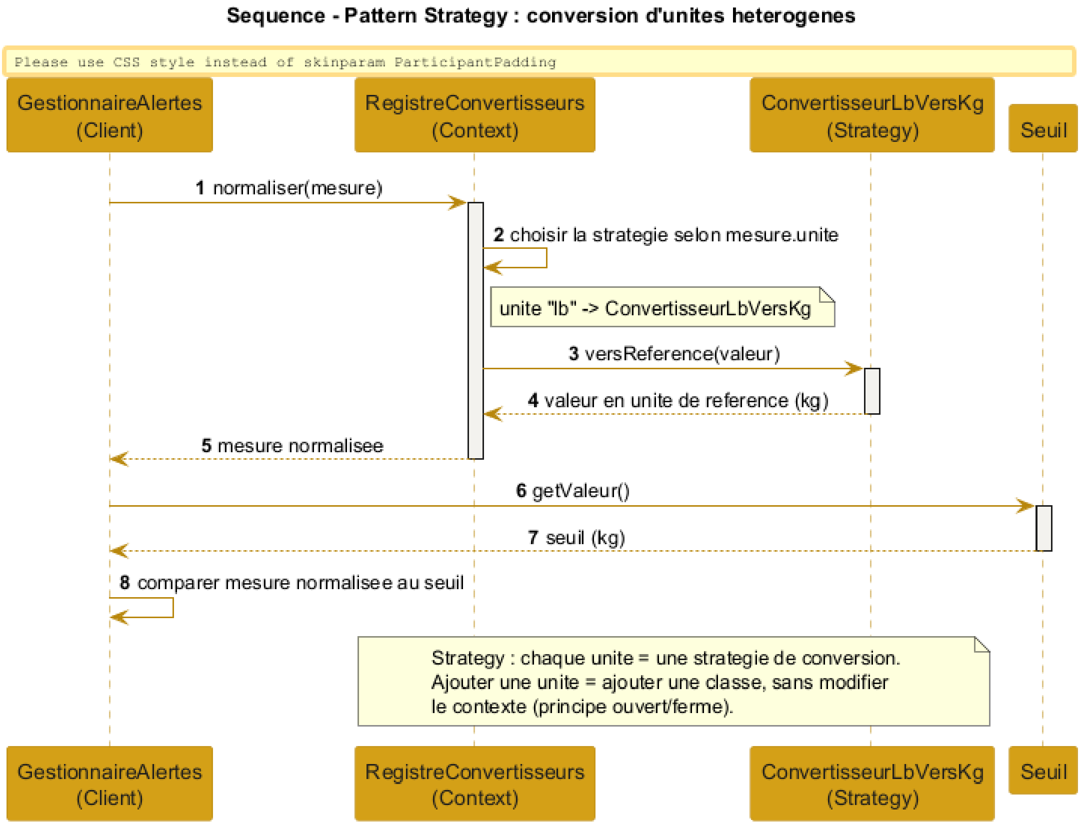

Figure E.6. Sequence of the Strategy pattern: conversion of heterogeneous units to the reference unit.

E.4.6 Adapter: ingestion of heterogeneous sensors

Figure E.7 illustrates the integration of a market sensor: the `AdaptateurBroodMinder` translates the sensor's proprietary API into a standardised `Mesure` (converted units, timestamp, geolocation), exposing the uniform `Capteur` interface expected by the business layer. The heterogeneous sensors thus become interchangeable.

E.5 Distribution of the patterns per layer

Figure E.8 summarises the location of each pattern in the multi-tier architecture. The *Builder* sits in presentation. The *Facade*, the *Proxy* and the *Command* queue belong to control. The domain (*Composite*, *State*, *Observer*, *Strategy*, *Factory Method*) and the segregated services (*ISP*) are placed in business. The *Repository* (*DAO / DIP*) and the *Adapter* handle data access. Finally, the configuration and internationalisation *Singleton* as well as the unit-conversion *Strategy* occupy the cross-cutting layer. This view confirms that the patterns reinforce the loosely coupled layered architecture without altering its structure.

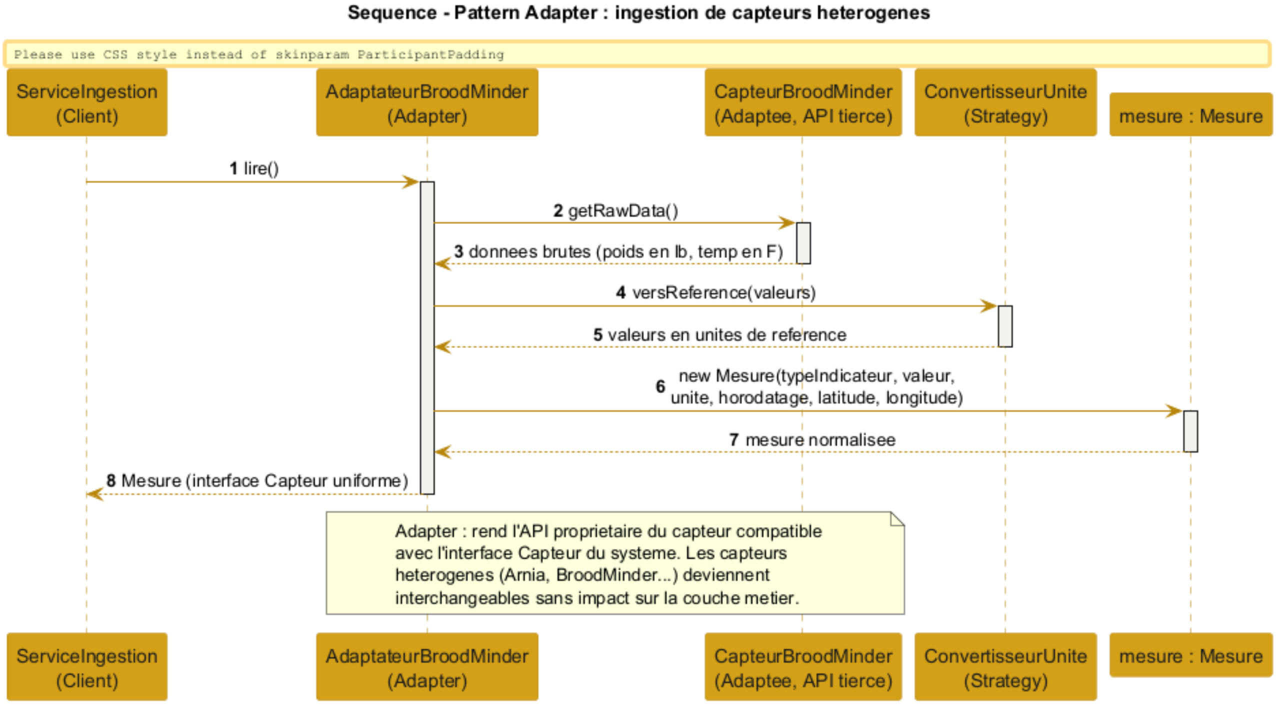

Figure E.7. Sequence of the Adapter pattern: ingestion of heterogeneous sensors (automated part).

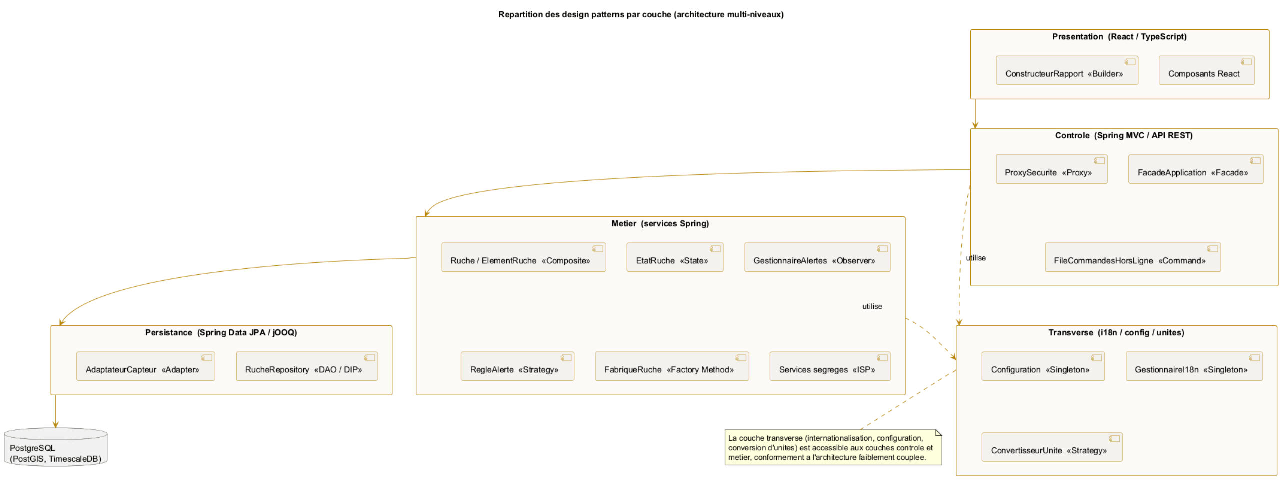

Figure E.8. Distribution of the design patterns per layer of the multi-tier architecture.

APPENDIX F

Appendix – Complements: data, UI, security and tests

This appendix gathers complementary deliverables reinforcing the file: the logical data model, the interface mockups, the access-rights matrix, the risk register, data-protection compliance and the test strategy.

F.1 Logical data model (LDM)

The LDM translates the data dictionary (chapter 5) into relational tables with primary keys, foreign keys and management constraints (`CHECK` on the number of supers, the frame capacity and the productivity, as well as the uniqueness of the compartment/slot pair). It distinguishes the *location* relationship (a site hosts hives) from the *ownership* relationship (a farm owns hives), in accordance with the rule allowing a site to host hives from several farms.

F.1.1 Vocabulary correspondence table

To guarantee consistency between the deliverables, each business entity carries the same concept in three forms: the **data dictionary** name (chapter 5), the UML diagram **class** and the LDM **table**. Attribute writing convention: `camelCase` in the classes (`nbHausses`), `snake_case` in the tables (`nb_hausses`).

Table F.1. Name correspondence between dictionary, classes and LDM.

Dictionary (business)	Class (UML)	Table (LDM)
Farmer	`Fermier`	`FERMIER`
Farm	`Ferme`	`FERME`
Site	`Site`	`SITE`
Hive	`Ruche`	`RUCHE`
Compartment (body / super)	`Compartiment`	`COMPARTIMENT`
Frame	`Cadre`	`CADRE`
Agent	`Agent`	`AGENT`
Schedule	`Planning`	`PLANNING`
Visit	`Visite`	`VISITE`
Photo	(visit report)	`PHOTO`

Dictionary (business)	Class (UML)	Table (LDM)
Measurement	`Mesure`	`MESURE`
Harvest	`Recolte`	`RECOLTE`
Threshold	`Seuil`	`SEUIL`

F.2 Interface mockups (wireframes)

The following low-fidelity mockups set the ergonomics of the key screens before development. They respect the visual identity and the required simplicity.

F.3 Access-rights matrix (RBAC)

The matrix specifies, for each function, the right granted to each profile. **Legend:** CRUD = create / read / update / delete, R = read, X = execute, A = approve, — = no access.

Function	Owner	Beek.	Sup.	Mgr.	Admin	API
Farms and sites	CRUD	R	R	R	CRUD	—
Hives (bodies/frames)	CRUD	R	R	R	CRUD	—
Agents	R	—	R	R	CRUD	—
Plan a visit	R	X	X	X	X	—
Approve the schedule	—	—	A	—	X	—
Carry out visit/report	R	X	X	—	R	—
Agents x hives calendar	R	R	R	R	R	—
Production dashboard	R	—	—	R	R	—
Alerts dashboard / handle	R	—	—	X	R	—
Summary and ROI	R	—	—	R	R	—
Configure thresholds/units	—	—	—	R	CRUD	—
Internationalisation	—	—	—	—	CRUD	—
CSV / TXT export	R	R	R	R	R	—
Backup / restore	—	—	—	—	X	—
Honey-quantity API	—	—	—	—	—	R

This matrix must be applied server-side (systematic rights checking in the control layer), and not only by hiding the interface elements.

F.4 Risk register

Risk	Prob.	Impact	Reduction measure
Development delay	Medium	High	Prioritise the CRUD core, with weekly milestones (W5 to W12).
Scope too broad	Medium	High	Limit to the high-priority functions, the rest optional.

Risk	Prob.	Impact	Reduction measure
Arabic i18n complexity (RTL)	Medium	Medium	Early end-to-end FR/EN/AR testing, `dir=rtl` handling.
Loss of field data	Medium	High	Offline command queue + synchronisation + backup.
Application vulnerabilities (injection, XSS)	Medium	High	Parameterised queries (JPA / jOOQ), React automatic escaping, security review.
False alerts (sensors)	Medium	Medium	Configurable thresholds + human validation through the report.
Google OAuth unavailability	Low	Medium	Local authentication fallback for the demonstration.
Measurement volume	Low	Medium	Archiving and aggregation per period (automated part).
Use of a code generator	Low	Very high	In-house design and code, traceability and peer review.

F.5 Protection of personal data

The system handles personal data (names and contacts of the farmers and agents) and potentially sensitive data (precise **geolocation** of the sites). The following principles are adopted, in the spirit of the GDPR:

- **Minimisation and purpose**: collect only the data useful for beekeeping monitoring.
- **Security**: encryption of exchanges (SSL / X.509) and authenticated access, already planned.
- **Access control**: strict application of the RBAC matrix above.
- **Individuals' rights**: viewing, rectification and **erasure** of the data, **portability** ensured by the CSV / TXT export.
- **Retention**: retention period defined for the measurements and visits, beyond which the data are archived or anonymised.
- **Traceability**: logging of access and sensitive actions (schedule approval, site closure).

The above principles translate into the following technical measures, by data category. **Encryption at rest** targets, as a priority, the sensitive columns (contacts, geolocation). **Pseudonymisation** dissociates the agent's identity from the agronomic history upon anonymisation.

Table F.4. Technical data-protection measures (spirit of the GDPR).

Data	Purpose	Retention	Security / erasure
Identity (name, contact)	Accounts and responsibilities	Relationship + 1 year	Encryption at rest, RBAC, anonymisation on request
Site geolocation	Monitoring, maps, weather	Site lifetime + 1 year	Encryption at rest, reducible precision, erasure at closure
Visits and reports	Health and production traceability	5 years	Encryption at rest, pseudonymisation (agent dissociation)

Data	Purpose	Retention	Security / erasure
Inspection photos	Visual monitoring of the colony	2 years	Encrypted storage, restricted access, deletion with the report
Sensor measurements	Performance analysis	Raw 1 year, then aggregated	Encryption at rest, aggregation / anonymisation
Access logs	Security and accountability	6 to 12 months	Administrator access, automatic purge

Erasure procedure: on request from a concerned individual, the system first produces an export (CSV / TXT portability), then erases or anonymises the records according to the table above. A **record of processing activities** and a data-protection referent are maintained for the project.

F.6 Test strategy

The strategy complements the functional test plan (chapter 10) with the unit and integration levels. The loosely coupled design (`Repository` interfaces, segregated services) makes the business rules testable in isolation, by injecting doubles (mocks).

Level	Scope	Examples	Tool
Unit	Isolated management rules	Frame/super constraints, honey calculation (Composite), ROI, thresholds	JUnit 5, Mockito
Integration	Data access (Repository / JPA)	Persisted CRUD, calculation queries	JUnit + Testcontainers
System	End-to-end functional cases	Cases TC01 to TC16 of chapter 10	JUnit, Selenium
Acceptance	Validation by the demonstration	Agent, farmer, manager journeys	Defence

- **Target coverage**: at least 70 % of the business layer (management rules), measured by `JaCoCo`.
- **Traceability**: each requirement is linked to at least one test case (matrix of chapter 10), completed by the corresponding unit tests.
- **Automation**: execution of the tests at each build (Maven), a prerequisite for delivery.

Synergy with the design: dependency inversion (DIP) makes it possible to substitute an in-memory `RucheRepository` for the JPA implementations, which makes the tests fast, deterministic and independent of the database.

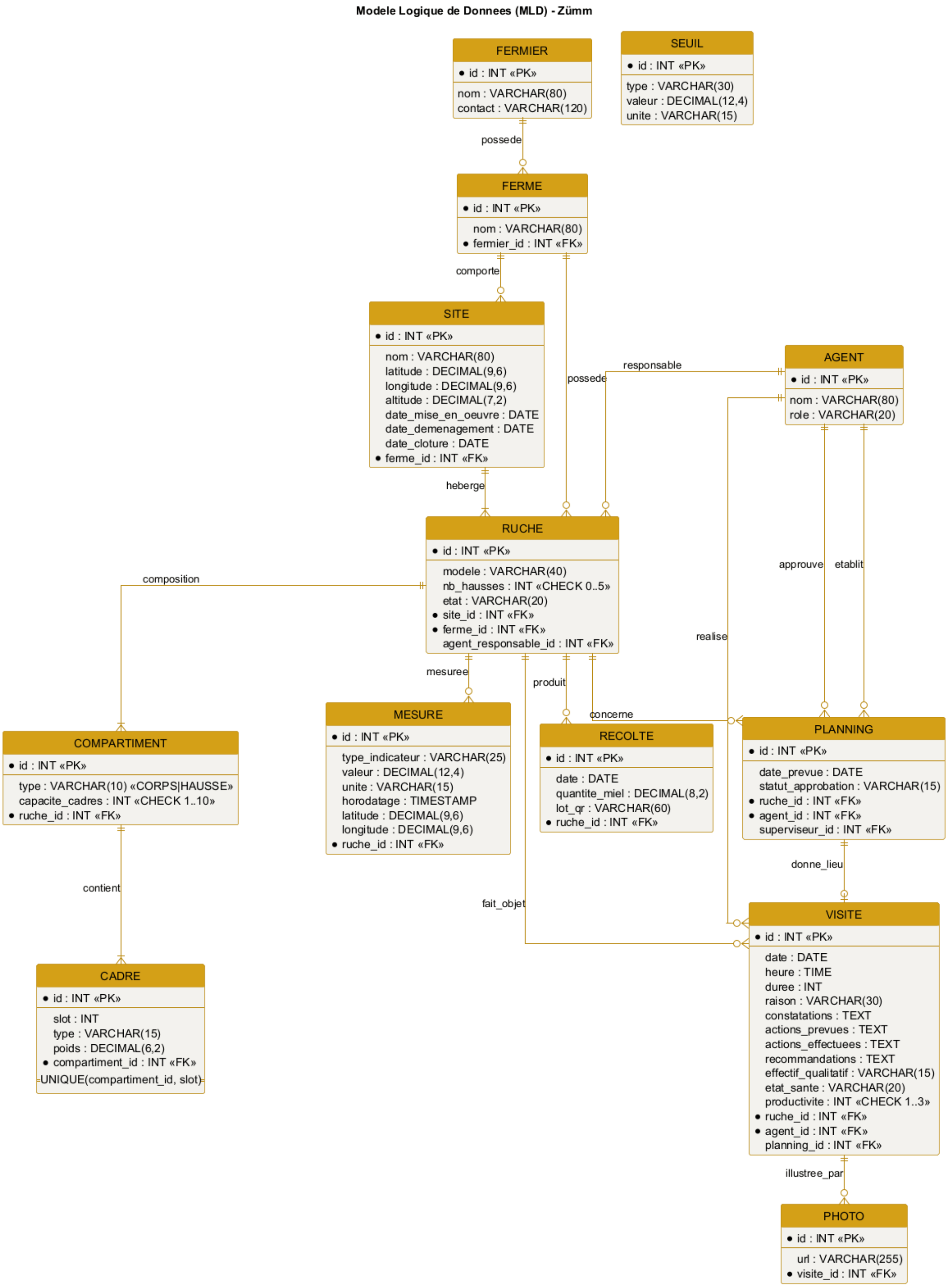

Figure F.1. Relational logical data model of the Zümm system.

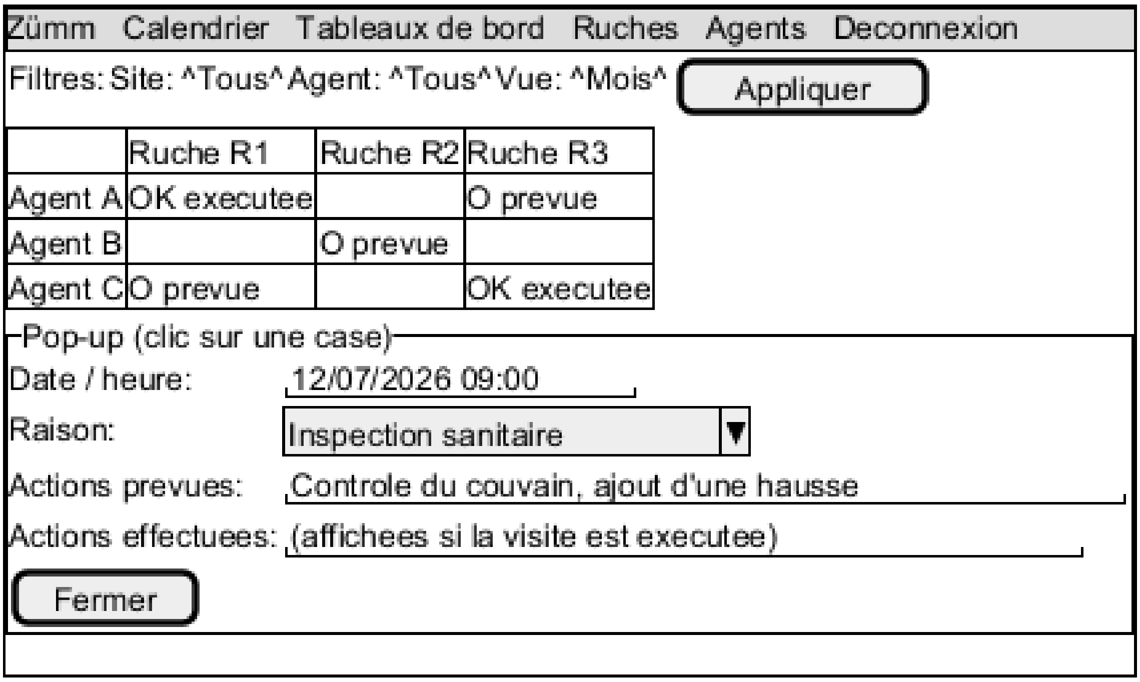

Figure F.2. Mockup: calendar / agents x hives matrix with visit pop-up.

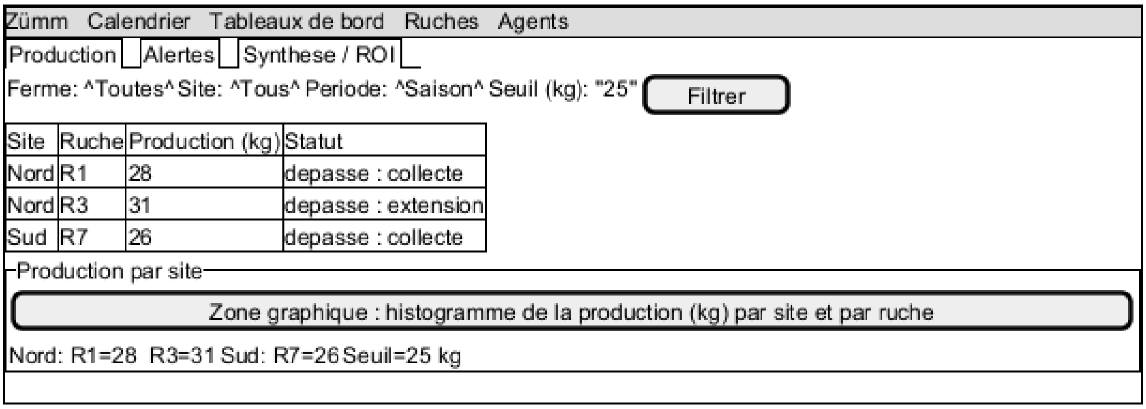

Figure F.3. Mockup: production dashboard with configurable threshold.

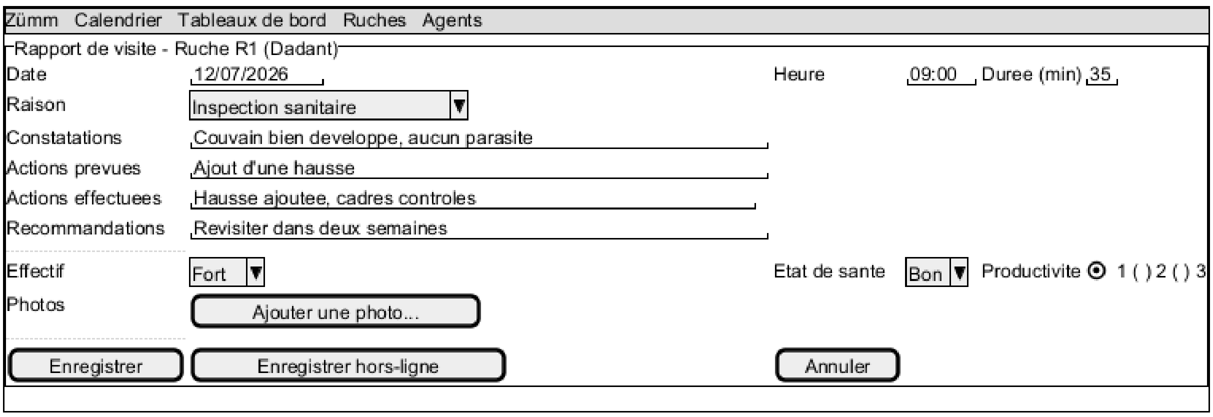

Figure F.4. Mockup: visit-report form (with offline mode and photos).

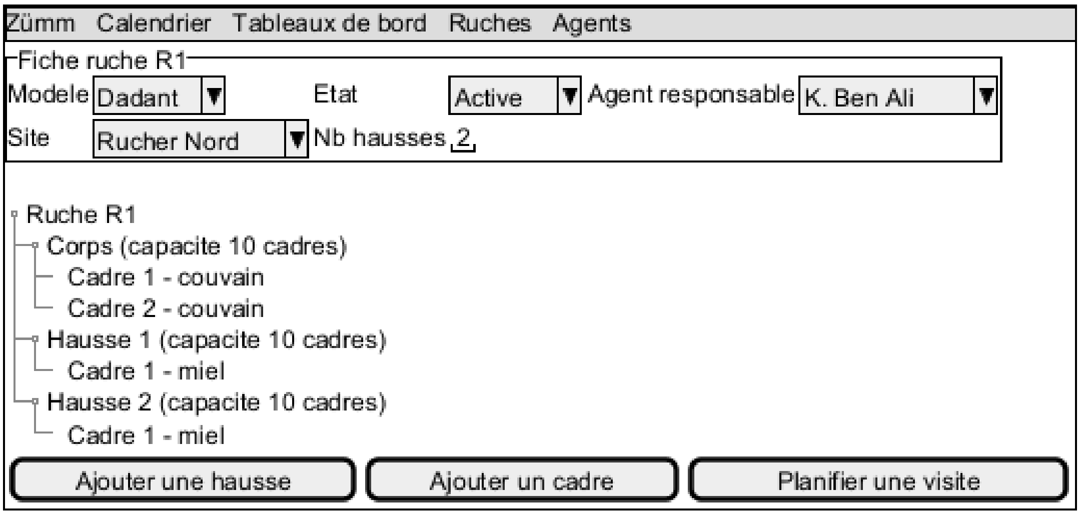

Figure F.5. Mockup: hive form and composition (body, supers, frames).

APPENDIX G

Appendix – Artificial intelligence and predictive analysis

This appendix answers the questions of the *why*, the *how* and the *where* of artificial intelligence in **Zümm**. It creates no new requirement: it extends the sensor part and the predictive analysis already anticipated in chapter 7 (§ 4.5) and builds on the technology stack adopted in appendix D. The objective is to document an AI integration **consistent with the loosely coupled architecture**, **without betraying the constraints** of the requirements specification (self-deployment, free of charge, human validation).

Positioning: in Zümm, AI is a **decision-support module**, never a control organ. It produces explainable *pre-alerts*; the beekeeping agent keeps the **final validation** through the visit report, in accordance with the evaluation rule of § 4.5.4.

G.1 Guiding principles

Every AI building block adopted respects four principles drawn directly from the project requirements:

- **Decision support and human validation**: AI flags, it does not decide. Each output is confirmed by the agent (§ 4.5.4).
- **Self-deployment without a mandatory third-party service**: no essential function depends on a paid cloud API. The basic processing runs **locally**; cloud use remains *optional* and enabled by configuration.
- **Decoupling and modularity**: AI is a **module enabled** via `ConfigZumm.ini`, isolated from the business core by an interface, in the spirit of loose coupling (appendix E).
- **Explainability and frugality**: **interpretable** methods (statistics, rules) and economical ones are favoured, rather than opaque models, to remain defensible and reproducible.

G.2 Mapping of AI uses

The table below lists the AI uses relevant to Zümm, distinguishing what is **in scope** (implementable in the present foundation) from what is **anticipated** (interface specified, later development under the sensor part).

Use	Description	Cloud dep.	Effort	Status
Adaptive anomaly detection	Per-hive baseline replacing the fixed threshold, to detect an unusual drift in production, population or temperature.	No	Low	**In scope**
Acoustic swarming detection	Analysis of the sound signature for a swarming pre-alert 1 to 5 days in advance (precision beekeeping, § 4.5.1).	No	High	Anticipated
Vision: disease detection	Help in identification (varroa, brood) from the inspection photos attached to the visit report.	No	High	Anticipated
Assisted voice entry	Dictation of the report, transcription feeding the report fields (§ 4.7, low priority).	Optional	Medium	Anticipated
Summary assistant	Natural-language summary of the dashboards (ROI, alerts) for the owner.	Optional	Medium	Optional

Prioritisation recommendation: deliver the **adaptive anomaly detection** as a real function (feasible within the imposed foundation, without cloud dependency), and **specify the other uses as anticipated interfaces**. This is the posture already adopted by the requirements specification for the sensor part, which preserves the document's consistency.

G.3 Selected case: adaptive anomaly detection

§ 4.5.4 defines **fixed thresholds** (for example internal temperature $< 32\,°C$ or $> 36\,°C$) and an anti-rebound rule. The limit of a fixed threshold is that it ignores the **specific behaviour of each hive** and its **seasonality**. The natural evolution consists in learning a **per-hive baseline** and measuring the deviation from this baseline, which detects the "relative drop $> 20\,\%$ vs previous period" adaptively.

G.3.1 Formalisation (without code)

For each (hive, indicator) pair, an **exponentially weighted moving average** (EWMA) μ_t and a dispersion estimate σ_t are maintained, updated at each new normalised measurement x_t (reference unit, § 5.3):

$$\mu_t = \alpha\, x_t + (1-\alpha)\, \mu_{t-1}, \tag{G.1}$$

$$v_t = \alpha\, (x_t - \mu_{t-1})^2 + (1-\alpha)\, v_{t-1}, \qquad \sigma_t = \sqrt{v_t}. \tag{G.2}$$

The **anomaly score** is the normalised deviation (sliding *z-score*):

$$z_t = \frac{|x_t - \mu_{t-1}|}{\sigma_{t-1} + \varepsilon}. \tag{G.3}$$

A **pre-alert** is raised only if $z_t > k$ (sensitivity) **persistently over N consecutive measurements** (hysteresis, as in § 4.5.4). This continuity with the existing rule is deliberate: an absolute threshold is replaced by a *learned relative* threshold, without changing the anti-rebound logic or the human-validation principle.

G.3.2 Parameters (all configurable via ConfigZumm.ini)

Parameter	Role	Effect of the setting
α **(smoothing)**	Weight of the recent measurement	The larger α, the faster the baseline reacts (and the more sensitive to noise).
k **(sensitivity)**	Threshold on the z-score	The smaller k, the more sensitive the detection (and the more false alerts increase).
N **(persistence)**	Number of consecutive measurements	The larger N, the fewer false alerts (at the cost of a slight delay).
Window / season	Reference history	Allows a per-season baseline (nectar flow, wintering).

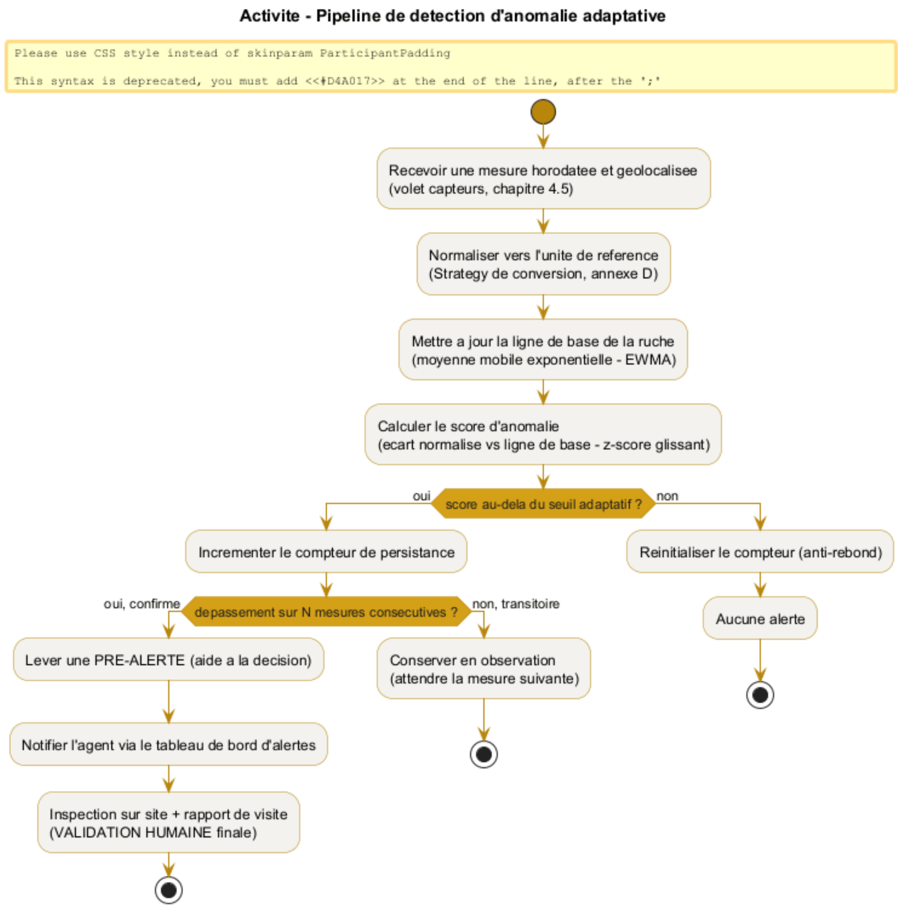

Figure G.1. Adaptive anomaly-detection pipeline, from the raw measurement to the pre-alert confirmed by the agent.

> **Consistency with § 4.5.4:** the value is first **converted to the reference unit** (conversion Strategy, appendix E), then compared with a threshold; the alert remains subject to **anti-rebound**

and remains a **decision-support aid**. Only the nature of the threshold changes: from *fixed* to *learned per hive*.

G.4 Integration architecture

AI fits in without breaking anything in the layered architecture (figure 7.2). Two decoupling mechanisms are used.

G.4.1 The detector as an interchangeable Strategy

Anomaly detection becomes a **`RegleAlerte` strategy** (Strategy pattern, appendix E), in the same way as the existing fixed threshold. The `GestionnaireAlertes` chooses, by configuration, between "fixed threshold" and "adaptive baseline" without its code changing (open/closed principle). AI is therefore not an invasive addition, but an **additional implementation of an already planned interface**.

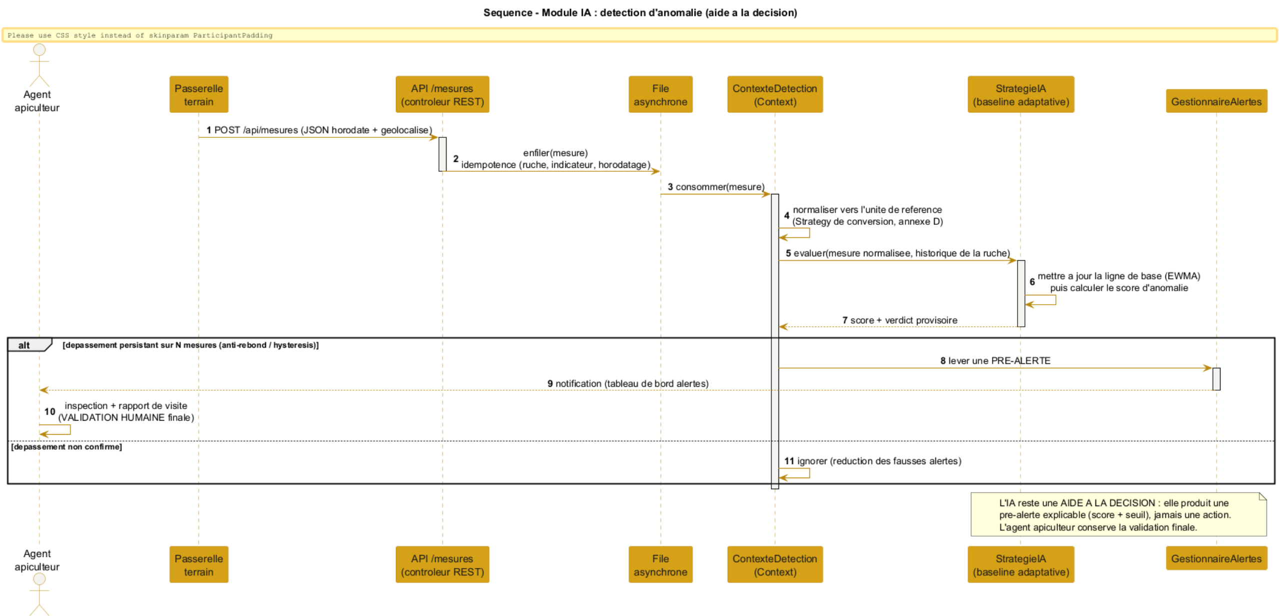

Figure G.2. Sequence of the AI module: the ingested measurement is evaluated by the adaptive strategy, which raises a pre-alert confirmed by the agent.

G.4.2 The heavy processing as a decoupled microservice

The uses requiring **signal or vision** (acoustics, photos) must **never** run in the main application process. They are offloaded to a **Python microservice** (pandas, scikit-learn / TensorFlow), exposed to the Java core by **REST/JSON**, in line with the API style adopted in appendix D. This microservice is **optional**: in its absence, the manual part and the basic statistical detection work fully.

G.5 Anticipated uses (specified interfaces, out of development scope)

In accordance with the treatment of the sensor part, the advanced uses are **specified as interfaces** so that the data model and the architecture accommodate them, without being developed in the present project.

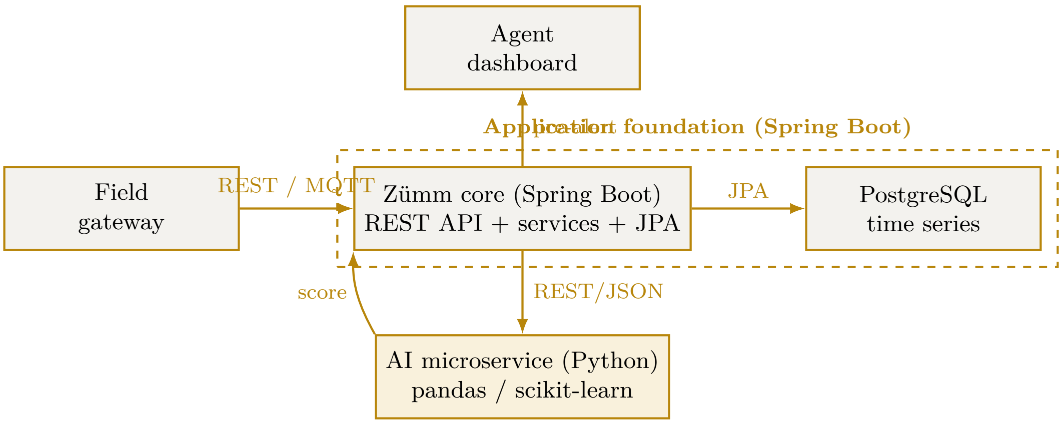

Figure G.3. Decoupled integration: the Spring Boot foundation remains in control, the AI microservice is an optional module plugged in via a web service.

- **Acoustics / swarming**: the data model already accommodates a `SIGNATURE_ACOUSTIQUE` indicator (§ 4.5). The AI microservice returns a verdict (swarming pattern detected, queen presence) that feeds a pre-alert.
- **Vision / diseases**: the **inspection photos** already attached to the visit report constitute the input set; a pre-trained model returns a suggestion, validated by the agent.
- **Voice entry**: the transcription can rely on the **browser's Web Speech API** (free, client-side, without server dependency), with an *optional* cloud fallback enabled by `ConfigZumm.ini`.

G.6 Ethics, limits and compliance

Issue	Adopted provision
False alerts	Hysteresis over N measurements and adjustable sensitivity k; the alert remains indicative.
Over-reliance on automation	Explicit positioning as decision support; the agent keeps the final validation (§ 4.5.4).
Explainability	Interpretable methods (score + threshold) preferred to opaque models; the pre-alert displays its justification.
Data protection	Local processing by default, SSL / X.509 encryption of exchanges, no imposed cloud transfer.
No lock-in	The AI microservice is optional and replaceable; without it, the basic management and detection remain operational.
Frugality	Priority to lightweight statistical processing; heavy inference is triggered only on demand.

Conclusion: Zümm's AI boils down to a clear rule — *learn each hive's normal, flag the deviation, let the human decide.* This approach adds predictive value while fully respecting the constraints of self-deployment, loose coupling and human validation of the requirements specification. It turns the *anticipated* predictive part into a **concrete and progressive** implementation trajectory.

APPENDIX H

Appendix – Security and robustness

Security is a **cross-cutting** requirement already present in the requirements specification: encryption of exchanges by **X.509 / SSL**, delegated authentication by **Google OAuth**, the **RBAC** rights matrix and **GDPR** compliance (appendix F). This appendix **consolidates** it into a coherent **defence-in-depth** strategy, then addresses **robustness** (resistance to load and to peaks), since *availability* is one of the three security properties. The good practices adopted follow the domain's reference guides (web-site hardening) and the free **k6** tooling for load testing.

> **CIA triad:** security aims at **Confidentiality** (encryption, access control), **Integrity** (input validation, prepared statements, backups) and **Availability** (anti-DDoS, rate limiting, load tests). No building block is a *mandatory* third-party service: each has a free, self-deployable equivalent (ModSecurity, Let's Encrypt, k6), in line with the self-deployment requirement.

H.1 Synthetic threat model

The table sets the usual risks of a web application against the measure adopted in Zümm.

Threat	Measure adopted in Zümm
Interception of exchanges	End-to-end TLS / SSL, X.509 certificates, HTTPS redirection and HSTS header.
Credential theft	Google OAuth (no stored password), two-factor authentication handled by the provider.
SQL injection	Parameterised queries (JPA and jOOQ, never concatenation), input validation and typing.
XSS (injected script)	Systematic output escaping, `Content-Security-Policy` header.
CSRF (forged request)	Per-session anti-CSRF token, `SameSite` cookies.
Session hijacking	`HttpOnly` and `Secure` cookies, session expiry and rotation (already § 7).
Privilege escalation	**RBAC** access control and least privilege (appendix F).
Denial of service (DDoS)	Rate limiting, web application firewall (WAF), asynchronous ingestion of measurements.
Data exposure	Encryption at rest, GDPR minimisation, encrypted backups.

H.2 Defence in depth

Security does not rely on a single barrier but on **successive layers**: if one gives way, the others hold. Figure H.1 projects these layers onto Zümm's architecture.

Perimeter: WAF, anti-DDoS, rate limiting, CDN (optional, deployable)

Transport: TLS / SSL, X.509 certificates, HTTPS redirection, HSTS

Application: OAuth2, RBAC, input validation, CSP headers, anti-CSRF

Data: prepared statements, encryption at rest, least DB privilege

Operations: updates, logging & monitoring, tested backups

Figure H.1. Defence in depth: five protection layers projected onto Zümm's architecture.

H.2.1 Perimeter and transport

At the edge, a **web application firewall** (WAF, for example ModSecurity) filters malicious requests and **rate limiting** absorbs peaks and abuse attempts. **Transport** encryption is already required: TLS / SSL, **X.509** certificates, forced redirection to HTTPS and the `Strict-Transport-Security` (HSTS) header to forbid any cleartext fallback.

H.2.2 Application

The application layer combines delegated **authentication** (Google OAuth, with two-factor handled by the provider), least-privilege RBAC **access control** (appendix F), **input validation**, **anti-CSRF protection** and **security headers** (`Content-Security-Policy`, `X-Frame-Options`, `X-Content-Type-Options`). Session management relies on `HttpOnly` and `Secure` cookies already specified in chapter 7.

H.2.3 Data and operations

Data access goes exclusively through **prepared statements** (anti-injection), a **least-privilege** database account and **encryption at rest** of sensitive data. In operations, regular **updates** of the server and dependencies, **logging** and **monitoring**, as well as **encrypted backups whose restoration is tested**, close the loop.

H.3 Hardening grid

The table lists web-site hardening good practices and specifies, for each, its **status** in Zümm: already required by the requirements specification or added by this appendix.

Practice	Implementation in Zümm	Status
HTTPS / TLS everywhere	X.509 certificates, HTTPS redirection, HSTS.	Already required

Practice	Implementation in Zümm	Status
Strong authentication	Google OAuth + provider two-factor.	Already required
Access control (RBAC)	Least-privilege roles and permissions.	Already required
Parameterised queries	Exclusively parameterised data access (JPA / jOOQ).	Already required
Web application firewall (WAF)	Edge request filtering (ModSecurity).	**Added**
Rate limiting / anti-DDoS	Absorption of peaks and abuse.	**Added**
Security headers	CSP, X-Frame-Options, X-Content-Type-Options.	**Added**
Anti-CSRF protection	Per-session token, SameSite cookies.	**Added**
Updates & patches	Up-to-date application server and dependencies.	**Added**
Logging & monitoring	Access and error traces, alerts.	**Added**
Encrypted backups	Regular backup + tested restoration.	**Added**
Anti-malware scanning	Periodic scan of repositories and uploads.	**Added**

H.4 Robustness: load and reliability testing (k6)

Since **availability** is a security property, the system must withstand the nominal load and **peaks**. We adopt **k6**, a **free**, scriptable load-testing tool that simulates *virtual users* (VUs) and checks objective **thresholds**.

H.4.1 Types of tests

Type	Objective
Load	Verify the behaviour at the expected load (nominal users and measurements).
Stress	Find the breaking point by increasing the load beyond nominal.
Spike	Simulate a sudden burst, for example a massive surge of sensor measurements.
Soak (endurance)	Detect memory leaks and degradation over a long duration.

H.4.2 Zümm-specific scenarios

- **Burst ingestion**: `POST /api/mesures` in batches (sensor part, § 4.5), to validate the asynchronous queue and idempotency.
- **Dashboards**: concurrent viewing of the production and alerts dashboards (aggregations, charts).
- **API web service**: repeated calls to `getZummHoneyActualQuantity` by third-party applications.

H.4.3 Thresholds and verdict

The test fails if the objective thresholds are not met, which points to a bottleneck to fix (pool size, index, cache, scaling). For illustration:

```
thresholds:
  request p95 latency         < 500 ms
  HTTP error rate             < 1 %
  /api/mesures ingestion      sustained nominal throughput without loss
```

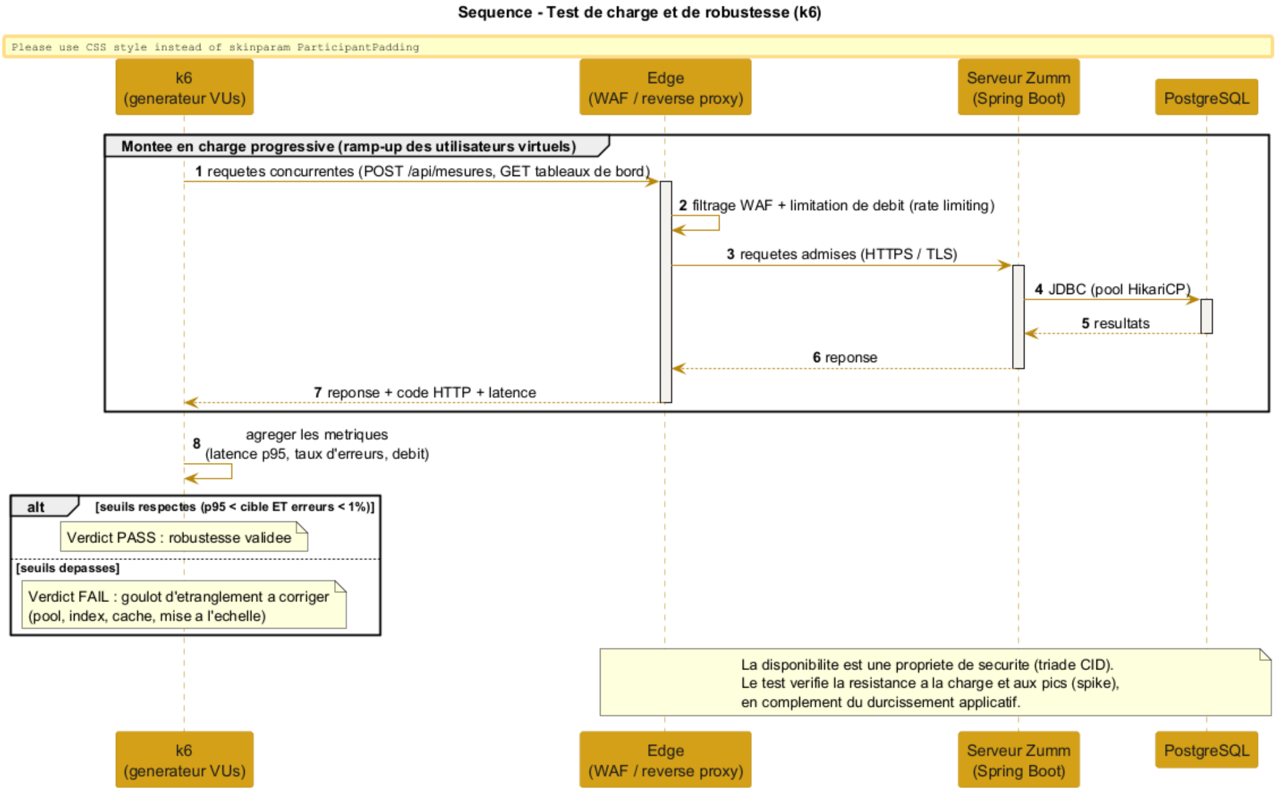

Figure H.2. k6 load test: progressive ramp-up, measurement of latency and error rate, verdict against the thresholds.

H.4.4 Complement to the test plan

The following cases extend the test plan of chapter 11 (TC01–TC16) on the security and robustness aspect.

ID	Object	Expected result
TC17	Load test (k6)	p95 and error-rate thresholds met at nominal load.
TC18	Spike test	The system absorbs a burst without loss or lasting error.
TC19	SQL injection / XSS	Malicious inputs rejected or neutralised (escaping).
TC20	Backup / restoration	Full restoration verified from an encrypted backup.

H.5 Pre-deployment checklist (go-live)

Before any production release, the points below are checked. This checklist summarises the previous sections into an operational review, domain by domain.

Domain	Points to check before production
Secrets & configuration	No cleartext secret in the repository; `ConfigZumm.ini`, keys and certificates out of version control, stored in a protected way.
Transport	HTTPS forced, cleartext-traffic redirection, HSTS header active, X.509 certificates valid and not expired.
Authentication & access	Google OAuth operational, RBAC matrix verified, default accounts and access disabled.
Application	Security headers (CSP, X-Frame-Options), anti-CSRF protection, input validation, error messages without information leakage.
Data	Prepared statements everywhere, encryption at rest of sensitive data, recent backup **and tested restoration**.
Dependencies	Application server and libraries up to date, dependency vulnerability scan performed.
Robustness	k6 load test **green** (p95 and error-rate thresholds), spike test passed, ramp-up and rollback plan defined.
Observability	Access and error logging, monitoring and alerts in place, consistent timestamping.
Compliance	GDPR respected (minimisation, retention period, rights), notices and register up to date (appendix F).

Release rule: an unmet point is **blocking** until fixed. The checklist is replayed at each major release, in the spirit of continuous improvement and under **human control**.

H.6 Compliance and consistency with the requirements

Conclusion: the appendix consolidates **layered** security (perimeter, transport, application, data, operations) and adds **measured robustness** through load tests. It extends, without contradicting, the requirements already set (X.509 / SSL, OAuth, RBAC, GDPR) and respects **self-deployment**: each building block adopted (ModSecurity WAF, Let's Encrypt certificates, k6) is **free and open source**. Security remains, like the alerts, a continuous process placed under **human control**.

APPENDIX I

Appendix: Operations and service commitments

A system delivered without a measurable service commitment cannot be operated: with no objective, no degradation can be qualified as an incident. This appendix sets the service objectives, the backup policy and the conditions of reversibility. It is the reference for the acceptance criteria of chapter 10.

I.1 Service level objectives (SLO)

The objectives are sized for a **single-server** deployment, in line with the operations decision adopted. They are deliberately realistic: promising 99.99 % without a redundant architecture would be an unkeepable commitment.

Indicator	Objective	Detail
Availability	99.5 % monthly	About 3 h 40 of tolerated downtime per month, excluding planned maintenance.
Planned maintenance	4 h/month maximum	Announced 7 days in advance, outside the production season where possible.
Latency (p95)	< 500 ms	On consultation screens, at nominal load.
Latency (p99)	< 1,500 ms	Tolerates heavy analytical queries.
Error rate	< 1 %	5xx responses over total requests.
RTO (recovery time)	< 4 h	Maximum delay to restore service after a major incident.
RPO (data loss)	< 1 h	Maximum acceptable loss, guaranteed by the backup frequency.

Reference nominal load: 200 concurrent users at seasonal peak, 10,000 instrumented hives, about 1 million measurements ingested per day. The latency objectives hold only within these limits; beyond them, the sizing must be revised.

I.2 Backup and restoration

I.2.1 Policy

- **Continuous backup** of PostgreSQL transaction logs (WAL archiving), guaranteeing the one-hour RPO.
- **Daily full backup**, encrypted at rest.
- **Off-site storage** on a medium separate from the production server — a backup stored on the machine it protects protects nothing.
- **Retention**: 7 daily backups, 4 weekly, 12 monthly.
- **Visit photographs** and the `ConfigZumm.ini` file are within the backup scope, on the same footing as the database.

I.2.2 Restoration test

> **A backup never restored is not a backup.** Full restoration is tested **monthly** on a dedicated environment, and the report is archived. A successful restoration test is a blocking acceptance criterion.

The test verifies three points: the integrity of the restored data, compliance with the announced RTO, and the ability of an operator to carry out the procedure using the runbook alone.

I.3 Monitoring and alerting

Signal	Alert threshold	Expected action
Application unavailable	2 consecutive failures	Immediate intervention
Disk space	$> 80\ \%$	Extension or purge within 48 h
Latency p95	$>$ 500 ms over 15 min	Analyse the offending query
5xx error rate	$> 1\ \%$ over 5 min	Immediate intervention
Backup failure	1 occurrence	Same-day intervention
TLS certificate	expiry < 30 d	Renewal
Sensor ingestion lag	> 1 h	Check the MQTT broker

Monitoring relies on **OpenTelemetry**, Prometheus and Grafana, in line with the stack of appendix D.

I.4 Reversibility

The client must be able to take the system back without depending on the supplier. The following are handed over at delivery, and on any later request:

- the **full data export** in an open, documented format (SQL and CSV), including photographs and attachments;
- the **source code** and its version history;

- the **operations documentation** (runbook): start, stop, restoration, certificate rotation, reading the alerts;
- the **database schemas** and migration scripts;
- the **redeployment procedure** from a clean repository, verified during acceptance.

Verification: reversibility is demonstrated, not declared. A clean redeployment from the repository, onto a fresh machine, is performed during acceptance.

I.5 Knowledge transfer

Two days of transfer are planned at delivery, covering routine administration, reading the monitoring, the restoration procedure and first-level troubleshooting. The runbook is the supporting material and remains the reference after the transfer.